

ABSTRACT

This thesis presents a major project on the design and implementation of an EEG-based emotion recognition system using the GAMEEMO dataset. The increasing demand for objective affective computing has led to a growing need for reliable systems that can interpret human emotions directly from brain activity. This project aims to develop a classification pipeline that can successfully recognize emotional states from EEG signals to facilitate more natural human-machine interaction.

The thesis provides an in-depth discussion of the neurobiological basis of emotions, the characteristics of EEG rhythms, and a review of deep learning methodologies for signal analysis. The proposed system is designed to process 14-channel EEG data and incorporates a variety of recurrent neural network architectures—including Baseline LSTM, Frequency-Band Attention LSTM, Dual-Stage Attention RNN, and xLSTM—to monitor and classify valence and arousal states.

The project involves the development of a comprehensive workflow, including signal preprocessing with Butterworth filters, feature extraction using Welch's Power Spectral Density, and the implementation of an ensemble framework for uncertainty filtering. The system is also tested and validated using Leave-One-Subject-Out (LOSO) cross-validation, and the results, highlighting the superior generalization of the xLSTM model, are presented and discussed in detail.

Overall, this thesis provides a comprehensive analysis of EEG-based emotion recognition and proposes a robust deep learning design for interpreting complex neural patterns. The project aims to improve the accuracy and interpretability of affective BCI systems, which is critical for the continued growth of mental health monitoring and adaptive technologies in the future.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
Chapter 1	Introduction.....	1
1.1	Overview of the Project.....	1
1.2	Motivation: The Quest for Objective Emotion Assessment.....	2
1.3	Past Work.....	3
1.4	Current Model.....	3
Chapter 2	Literature Survey.....	5
2.1	Brain Regions Implicated in Emotion Processing.....	5
2.2	EEG Rhythms and Emotional States.....	7
2.3	Characteristics and Challenges of EEG Signals.....	8
Chapter 3	Methodologies for EEG-Based Emotion Recognition.....	11
3.1	EEG Data Acquisition: Hardware and Electrode Placement.....	11
3.2	Signal Processing: Enhancing Signal Quality.....	13
3.3	Feature Extraction: Translating Brainwaves into Data.....	14
3.4	Comparative Analysis of Machine Learning Strategies.....	16
3.5	EEG Emotion Recognition Dataset.....	19
3.6	Model Evaluation and Performance Metrics.....	21
Chapter 4	Algorithm & Results.....	25
4.1	Algorithm.....	25
4.2	Mathematical Model: EEG Emotion Recognition.....	28
4.3	Model-Wise Results Analysis.....	29
4.4	Model Performance Comparison under LOSO.....	42
Chapter 5	Conclusion and Future Scope.....	43
5.1	Conclusion	43
5.2	Future Scope	43
	References.....	44
	Appendix.....	47

LIST OF FIGURES

Figure No.	Title	Page no.
Fig. 3.1	Workflow of an EEG-Based Emotion Recognition System.....	11
Fig. 3.2	Placement of Electrodes (10-20 System).....	12
Fig. 3.3	Band-Pass Filter.....	14
Fig. 4.1	Visualises the training and validation loss.....	30
Fig. 4.2	Baseline_LSTM accuracy.....	31
Fig. 4.3	Mean Frequency-Band Attention by Emotion Class.....	32
Fig. 4.4	FreqAtt_LSTM: Fold 1 Training Dynamics.....	33
Fig. 4.5	Band Attention Profiles per Emotion Class.....	34
Fig. 4.6	DA_RNN: Fold 1 Loss.....	35
Fig. 4.7	DA_RNN: Fold 1 AccuracyLoss.....	36
Fig. 4.8	DA-RNN Confusion Matrix.....	37
Fig. 4.9	xLSTM Training Dynamics(Loss).....	38
Fig. 4.10	xLSTM Training Dynamics(Accuracy).....	39
Fig. 4.11	xLSTM Confusion Matrix	40
Fig. 4.12	Confidence Threshold vs Accuracy/Coverage.....	41

LIST OF TABLES

Table No.	Title	Page no.
Table 3.1	Comparison of EEG Artifact Removal Techniques.....	18
Table 3.2	GAMEEMO Dataset Summary.....	20
Table 4.1	Baseline_LSTM Summed Confusion Matrix.....	30
Table 4.2	Mean Frequency-Band Attention Weights by Emotion Class.....	31
Table 4.3	DA_RNN Summed Confusion Matrix.....	35
Table 4.4	xLSTM Summed Confusion Matrix.....	38
Table 4.5	Ensemble Uncertainty Summary (LOSO Mean).....	41
Table 4.6	Comparative Mean Performance.....	42

LIST OF ABBREVIATIONS

Abbreviation	Full Form
EEG	Electroencephalogram
ECG	Electrocardiogram
EMG	Electromyography
EOG	Electrooculography
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
ANN	Artificial Neural Network
CNN	Convolutional Neural Network
SVM	Support Vector Machine
KNN	K-Nearest Neighbors
RF	Random Forest
HRV	Heart Rate Variability
PSD	Power Spectral Density
ERP	Event-Related Potential
ICA	Independent Component Analysis
PCA	Principal Component Analysis
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
FFT	Fast Fourier Transform
STFT	Short-Time Fourier Transform
MFCC	Mel-Frequency Cepstral Coefficients
BVP	Blood Volume Pulse
GSR	Galvanic Skin Response
CV	Cross Validation
RMSE	Root Mean Square Error
MAE	Mean Absolute Error
API	Application Programming Interface
IoT	Internet of Things
BCI	Brain-Computer Interface

CHAPTER 1

INTRODUCTION

Emotion recognition using Electroencephalography (EEG) is an emerging, fast-growing research area at the intersection of affective computing, neuroscience, and engineering. It is also an important part of Brain-Computer Interface (BCI) systems, which aim to understand human emotions directly from brain activity. Unlike conventional techniques, EEG systems measure brain electrical activity. There are no invasive procedures or pain involved, and sensors are placed on the scalp to pick up this activity. By analysing these signals, we can detect subtle variations in brain activity that correspond to different emotions.

The main aim of this project is to design a system that can successfully recognize human emotions from EEG signals. This will facilitate machines to interact with humans in a more natural, smart and understanding way. Such systems are an important part of affective computing, which focuses on helping machines understand and react to human emotions.

1.1 Overview of the Project

This report is a systematic investigation of EEG-based recognition of emotion using the GAMEEMO dataset [1], and a set of recurrent neural models, which are designed to be applied to frequency-band features. The task is to assess model performance and generalization on naturally elicited emotions in interactive games, and to give interpretable information on what spectral bands and channels give most of the contribution to classification decisions. Our pipeline will include data acquisition, signal preprocessing and band-power extraction, sequence construction, model training and evaluation, and explainability analysis. In particular, we train and compare four model families: a Baseline LSTM, a Frequency-Band Attention LSTM (FreqAtt LSTM) that learns the importance of each band, a Dual-Stage Attention RNN (DA-RNN) that applies feature and temporal attention and an xLSTM variant that enhances the stability and generalization. We also test an ensemble which combines the outputs of the models to enhance robustness in mixed-subject experiments. Subject-independent Leave-One-Subject-Out (LOSO) cross-validation and stratified k-folds splits are used to evaluate the models and report generalization to unseen individuals as well as pooled performance. The research consists of

explainability and uncertainty: we compute band-level SHAP summaries and predictive entropy statistics to characterize model confidence and to highlight those frequency bands that most strongly affect each emotion category. All code, preprocessing parameters, and artifacts of results are stored in the repository and are referenced throughout the report to aid in reproducibility. The rest of the chapter summarises motivation and background to it; Chapter 2 gives a review of past work; Chapter 3 details methodology and models, Chapter 4 presents code and empirical results, and finally we discuss implications, limitations and future directions.

1.2 Motivation: The Quest for Objective Emotion Assessment

The use of EEG for emotion recognition is mainly due to the problems in traditional methods. Common methods like facial expressions, voice tone, body language, and questionnaires are often not very reliable. These methods are subjected to many external factors like culture, social environment, age and personal style of expressing emotions. Moreover, people can hide or fake their emotions which makes it hard to know what they are really feeling. These methods are also of little use in situations such as patients who are unable to communicate properly or people with disabilities.

Self-report methods also have problems of bias, lack of awareness and difficulty to explain feelings clearly. But EEG provides a more straightforward and objective way of understanding emotions. It captures brain activity as it occurs spontaneously and without the ease of manipulation. Hence, EEG signals are perceived to be more reliable in evaluating real emotional states.

EEG based emotion recognition can be very useful in cases such as: Monitoring mental health of non-communicative patients Measuring stress and mental workload Detecting emotional conditions in critical situations like driving or defense systems Another advantage of EEG is its non-invasive and easy to use nature. With availability of portable and affordable EEG devices, it becomes more feasible for real-life use.

1.3 Past Work

Public datasets, including DEAP [2] and AMIGOS [3], have fueled much of the early progress including provision of standardized stimuli (images, music or video) and physiological recordings and self-report labels. These datasets stress on controlled and repeatable paradigms but typically rely on passive stimuli that are not necessarily reflective of affective dynamic occurring during naturalistic interactions. GAMEEMO compliments these corpora eliciting affect through interactive game play with greater ecological validity and lower distance/challenges/location label granularity and motion artifacts.

Past research demonstrates that when frequency and selection of bands (e.g., an emphasis on the beta/gamma band or a mix of bands) is carefully chosen and selected, improvement of performance can be achieved but results not predictable across all datasets and recording hardware. Previous methods employed classical classifiers on hand-engineered features [4] (SVMs, random forests). The most recent surge to dominate is based on the capabilities of the deep learning methods, specifically recurrent networks (LSTM/GRU) which are capable of directly learning temporal and spatial patterns.

1.4 Current Model

The raw EEG recordings of the Emotiv Epoc+ (14 channels) are bandpass filtered to 4-45 Hz and divided into overlaying time windows (see parameters in the code). On every window we calculate Welch PSD and sum up the power over five canonical bands (delta, theta, alpha, beta, gamma). Per-channel band powers are next averaged or aggregated into a sparse representation in such a manner that each time step is represented in a 14x 5 matrix (channels x bands). Sequences of fixed length (seq_len) are built with a sliding window and a configurable amount of overlap and stride. Scaling/augmentation of features: All band features are standardized based on the statistics of their training-folds to prevent leakage between training-folds. Temporarily restricted augmentations are optional, including sequence subsampling and limited time-warping; these are also set conservatively due to the rather small number of subjects. Model family details: - Baseline LSTM: Two stacked layers of LSTM (64 $\rightarrow$ 32 units) with dropout and final dense classifier. Is an ablation baseline of the temporally modeling without attention. - Frequency-Band Attention LSTM

(FreqAtt LSTM) A lightweight attention-based module that estimates band importance (via softmax) using band features pooled across time, reweighting the band features of all time-steps, and feeds the attended sequence through an LSTM encoder. An auxiliary attention-extractor model is used to generate weights used in interpretation. - Dual Stage Attention RNN (DA-RNN): Stage-1 feature attention- Stage-1 features It computes per-time-step importance over bands/channels; Stage-2 temporal attention- Stage-2 encoder states It computes importance per-band/channel at each time-step; Stage-2 temporal attention- Stage-2 encoder states It computes importance per-band/channel at each time-step; This architecture aims at the spatial/spectral and temporal salience. - xLSTM (Extended LSTM): Upgraded recursive by blocks with layer normalization, gating and residual refinements to enhance gradient flow and generalization; tuned to ruggedness in LOSO tests. - Ensemble: Averaging over probabilities of tuned model checkpoints (typically the four families above) to reduce variance and gain improved coverage in mixed-subject scenarios.

Chapter Summary

This chapter introduced EEG-based emotion recognition as an emerging interdisciplinary field at the intersection of engineering, neuroscience, and affective computing. The motivation for using EEG over conventional emotion-detection methods, such as facial expressions, vocal tone, and self-report surveys, was established: EEG captures involuntary neural activity that is difficult to mask, providing an objective window into emotional states. The chapter highlighted the growing accessibility of affordable wearable EEG devices and the parallel advances in deep learning algorithms as key drivers pushing this field from specialised laboratories into real-world applications.

CHAPTER 2

LITERATURE SURVEY

AMIGOS: A Dataset for Affect, Personality and Mood Research on Individuals and Groups by Miranda-Correa et al. proposes a multimodal dataset [5] for the study of emotional responses of individuals and groups. The dataset consists of EEG and other physiological signals recorded while participants watched audiovisual stimuli, and their self-reported ratings. The authors showed that the inclusion of social context and group interactions gives a better understanding of affective computing and improves the analysis of emotion recognition .

The paper Investigating Critical Frequency Bands and Channels for EEG-Based Emotion Recognition with Deep Neural Networks by W.-L. Zheng and B.-L. Lu, analyzed the importance of different EEG frequency bands and brain regions for emotion classification. The research found that the Gamma and Beta [6] bands of frequency were most important for emotional information and the use of deep learning models on extracted features improved the classification performance and stability across sessions.

Zhang et al. proposed a novel recurrent neural network architecture, termed as spatial-temporal recurrent neural network [7], for emotion recognition that captures both spatial and temporal dependencies in the EEG signals. The model combines spatial relationships between EEG channels and temporal dynamics over time, thus resulting in improved accuracy compared to traditional models. The study showed that combining spatial and temporal modelling results in more effective emotion recognition systems.

2.1 Brain Regions Implicated in Emotion Processing

To interpret the EEG signals for emotion recognition, it is important to understand the neural basis of emotion. Neuroscience studies have shown that emotions are not located in a specific area of the brain, but are a result of complex interactions between distributed neural networks in the cortical and subcortical areas [8]. Although EEG mainly measures electrical activity in the cerebral cortex, especially the synchronous activity of large groups of pyramidal neurons, this activity in the cortex is strongly affected by and interacts with deeper brain areas, which are critical in emotional

processing.

Key brain regions [9] consistently implicated in emotion include:

- **Amygdala:** A subcortical structure in the limbic system known to process fear and threat-related stimuli, but now known to be involved in processing both positive and negative valence. It is a key node in the assessment of the affective valence of sensory stimuli.
- **Prefrontal Cortex (PFC):** The prefrontal cortex is a large part of the cortex, with areas such as the orbitofrontal cortex (OFC), and ventromedial PFC (vmPFC) of particular importance for the integration of emotion information with cognition, decision making, emotion regulation and the representation of stimulus value. Frontal EEG activity is often a key area of interest in studies of emotion.
- **Anterior Cingulate Cortex (ACC):** Involved in monitoring conflict and plays a key role in detecting errors and conflicts, as well as combining emotional and cognitive data to direct actions.
- **Insula:** Insula plays a key role in sensing internal bodily states and shaping emotional experiences, especially empathy and disgust.
- **Temporal Lobe:** Hippocampus (involved in memory formation, often in the context of emotion) and superior temporal gyrus (auditory and social cue processing including emotional prosody).
- **Parietal Lobe:** Involved in attention and facilitates attention, sensory integration, and spatial awareness, thereby helping to contextualize emotional experiences.
- **Occipital Lobe:** visual processing, modulated by the emotional relevance of visual stimuli. Given the distributed nature of this network it is likely that full emotion recognition requires analysis of EEG signals from several locations on the scalp to capture coordinated activity across these involved regions. Sole reliance on activity from a single region, e.g. frontal asymmetry, may provide incomplete information about the complex emotional state.

Because this network is distributed, accurately recognizing emotions probably involves examining EEG data from various scalp sites to detect the synchronized activity among these regions. Depending exclusively on activity from one area, like

frontal asymmetry, might yield an incomplete picture of complex emotional states [12].

2.2 EEG Rhythms and Emotional States

EEG signals are composed of oscillatory activity across different frequencies, also termed brain rhythms or bands. Such rhythms are considered to reflect different states of neuronal synchrony and inter- and intra-regional communication, and their features are modulated by cognitive and emotional states [14]. The standard frequency bands and their general association, particularly relevant to emotion, are:

- i. **Delta (δ , 0.5-4 Hz):** Typically associated with deep sleep in adults but also observed during wakefulness in some cognitive tasks and emotional processing. Greater delta synchronization has been associated with emotional stimuli compared to neutral stimuli, possibly reflecting general cortical modulation or deep processing.
- ii. **Theta (θ , 4-8 Hz):** These are frequently associated with drowsiness, memory functions, and emotional states.
- iii. **Alpha (α , 8-13 Hz):** The dominant rhythm of relaxed, awake adults with closed eyes, usually most pronounced in posterior areas. Alpha power is often inversely related to cortical activation (event-related desynchronization, ERD, indicates activation). A key finding in emotion research is frontal alpha asymmetry: relatively greater left frontal alpha power (less activation) is often associated with positive valence/approach motivation, while greater right frontal alpha power (less activation) is associated with negative valence/withdrawal motivation.
- iv. **Beta (β , 14-30 Hz):** These are related to active thinking, alertness, concentration and motor control. In emotion literature, beta activity has been associated with arousal levels and may participate in resolving conflict between sensory input and cognitive processes. Increased beta responses are commonly seen for negative-valence and high-arousal stimuli.
- v. **Gamma (γ , >30 Hz):** Higher-cognitive functions, feature binding, attention, perception and consciousness. Increased gamma responses have been linked to early sensory processing of emotional stimuli and increased attention, particularly for negative-valence faces. Analysis of the power, frequency, and

spatial distribution (e.g., frontal asymmetry) of these rhythms provides important features for machine learning models to classify emotional states

Examining the intensity, frequency, and spatial patterns (such as frontal asymmetry) of these rhythms yields essential data for machine learning models aiming to categorize emotional states.

2.3 Characteristics and Challenges of EEG Signals

EEG signals are valuable for emotion recognition, but they come with their own complexities. It captures rapid neural dynamics, making it a treasure for analytics, where this analysis can be limited by signal quality and variability. The key strengths and challenges for EEG signals are outlined below.

2.3.1 Strengths:

- **Excellent Temporal Resolution:** High Temporal Resolution EEG can record the neural activity in the millisecond range, which allows studying the fast dynamics in emotional responses.
- **Non-Invasiveness:** EEG is measured with electrodes placed on the surface of the scalp, which is safe and can be applied with different populations including vulnerable groups.
- **Relatively Low Cost and Portability:** EEG systems are generally less expensive and more portable than other neuroimaging techniques such as fMRI or MEG, easing the conduction of research outside of specialized labs and the development of wearable applications. This is a key advantage over methods with a lower temporal resolution such as fMRI [13] .

2.3.2 Challenges:

- **Low Signal-to-Noise Ratio (SNR):** Brain generated electrical signals are very weak (in the microvolt range) and get attenuated as they pass through the skull and scalp. They are prone to contamination from noise originating from various sources.
- **Susceptibility to Artifacts:** EEG recordings are highly vulnerable to contamination from non-neural electrical artifacts, which frequently exhibit significantly greater amplitudes than the target brain signals.
- **Non-Stationarity:** The term non-stationarity refers to the temporal variation

in EEG signal statistics, which mirrors shifts in brain states and cognitive functions. This breaches the stationarity premise that certain traditional signal processing methods depend on

- **Non-Linearity:** EEG signals have non-stationary statistical properties, i.e., the properties of EEG signals are changing over time continuously. This behaviour is based on the dynamic nature of the brain activity and the continuous cognitive and emotional processes. This violates the stationarity assumption required by some classical signal processing techniques
- **Inter-Subject Variability:** EEG signals can show substantial differences between individuals, even when they are exposed to the same stimulus, due to differences in anatomy, physiology, psychological characteristics and current state .

These intrinsic flaws go beyond simple technical hurdles, as they fundamentally define the domain of EEG-based emotion recognition. This requires creating advanced signal processing workflows that incorporate artifact elimination techniques and sophisticated feature extraction approaches designed to manage noisy, non-stationary, and non-linear data. Consequently, these obstacles encourage the use of robust machine learning methods, especially deep learning, which outperform basic linear models at identifying significant patterns in noisy, variable data. The superior temporal resolution remains a primary advantage, indicating that analytical methods utilising this time-based data are well-suited to capture the dynamic emotional processes evident in EEG.

Chapter Summary

This chapter surveyed existing literature on EEG-based emotion recognition, covering important datasets including AMIGOS and recent work applying spatial-temporal neural networks to emotion classification. The neural underpinnings of emotion were examined in detail: key brain regions (amygdala, prefrontal cortex, anterior cingulate cortex, insula, temporal lobe, parietal lobe, and occipital lobe) and their roles in emotional processing were described. EEG frequency bands and their associations with emotional states were outlined Delta (δ , 0.5–4 Hz) linked to deep sleep and baseline affect, Theta (θ , 4–8 Hz) to drowsiness and memory, Alpha (α , 8–13 Hz) to relaxation and frontal asymmetry, Beta (β , 14–30 Hz) to active cognition and anxiety,

and Gamma (γ , >30 Hz) to high-level processing and arousal. The chapter also discussed the key challenges of EEG-based systems: low signal-to-noise ratio, motion and electrode artifacts, non-stationarity, non-linearity, and significant inter-subject variability.

CHAPTER 3

METHODOLOGIES FOR EEG-BASED EMOTION RECOGNITION

The process of creating an EEG-based emotion recognition system generally follows a series of stages, beginning with emotion induction and data collection, proceeding through comprehensive signal processing, and concluding with feature engineering and selection. The intricate nature of this pipeline underscores the difficulties involved in deciphering erratic and inconsistent EEG data.

3.1 EEG Data Acquisition: Hardware and Electrode Placement

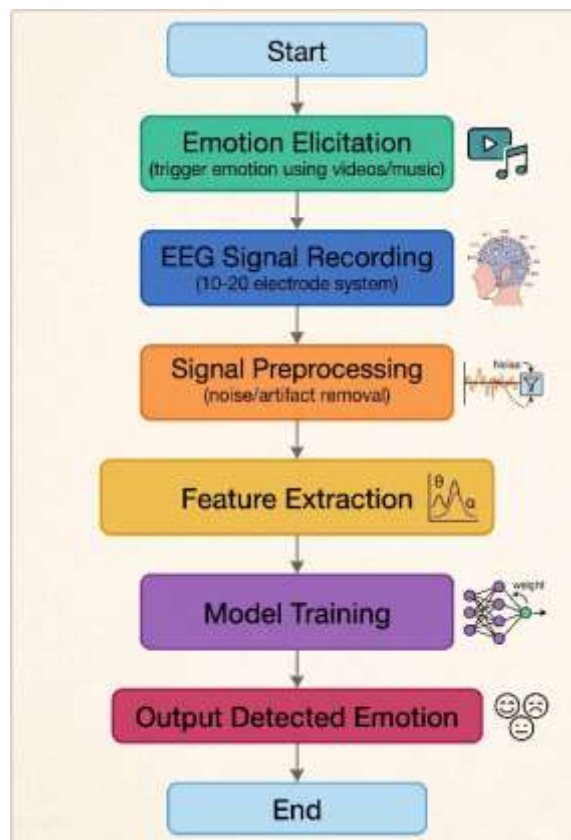


Fig. 3.1 Workflow of an EEG-based emotion recognition system

The quality and features of the collected EEG data are largely determined by the recording equipment and electrode configuration. The complexity of this pipeline highlights the difficulties in interpreting. The Fig. 3.1 captures the gist of the workflow in a flow chart.

3.1.1 EEG Devices: EEG devices range from high-density clinical and research

systems to many channels (64, 128 or more) and better signal quality to affordable portable consumer headsets, with fewer channels (generally 4 to 32), and a more straightforward setup [4]. Research-grade systems provide a more detailed data set, but the widespread availability of consumer devices allows wider application research.

3.1.2 The International 10-20 System: The International 10-20 system is a commonly used system for locate sensors on the skull standardize and reproduce electrode placement landmarks, accounting for differences in head size and shape

Landmarks and Measurements: Important reference points include the nasion, inion (the occipital protuberance), and preauricular landmarks. The distances between these landmarks are measured to establish electrode locations at 10% or 20% intervals along the sagittal and coronal axes, as well as around the skull's circumference.

Labelling Convention: Electrode nomenclature combines a letter denoting the brain area (Fp, F, C, P, T, O) with a number specifying the hemisphere as seen in Fig. 3.2, where odd digits represent the left side and even digits the right. Midline electrodes are designated with the suffix 'z' (zero) rather than a numerical value, as seen in examples like Fz, Cz, and Pz. Reference electrodes are frequently positioned on the earlobes or mastoids.

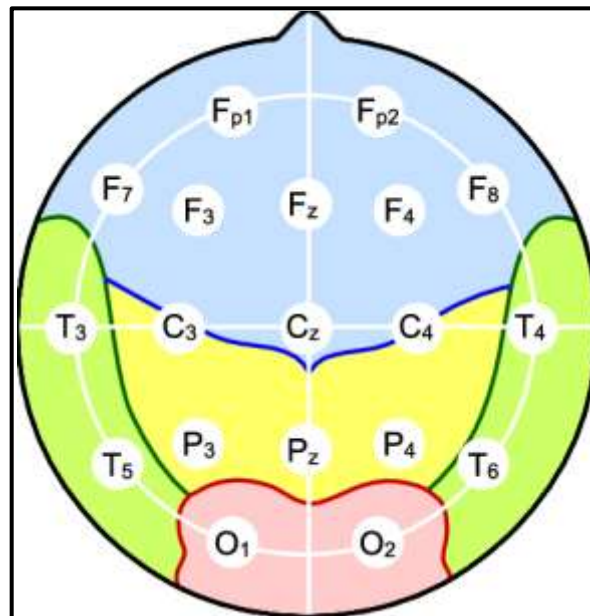


Fig. 3.2 Placement of Electrodes [5]

3.1.3 Electrode Considerations: The number of electrodes used affects the spatial data collected. Increased density gives more detail, but research suggests

that performance may flatten out, with good accuracy often reached with just 10 to 12 channels well-located. The type of electrode is also important, with traditional wet electrodes provide improved signal quality and lower impedance at the cost of comfort and setup time and convenience of dry electrodes, which allow quicker application but often have a higher impedance and are more sensitive to motion artifact.

The first stage of the system consists of the acquisition of the EEG data while the subjects are presented with emotionally-relevant stimuli. The project uses interactive gaming environments to provoke natural, dynamic emotional responses, moving away from traditional methods that rely on passive stimuli such as images or videos.

The study utilizes the GAMEEMO dataset, comprising EEG data from 28 participants. Every participant engages in gaming sessions crafted to elicit four unique emotional states:

- **Boring** (low arousal, low valence)
- **Calm** (low arousal, high valence)
- **Horror** (high arousal, low valence)
- **Funny** (high arousal, high valence)

3.2 Signal Processing: Enhancing Signal Quality

Raw EEG signals are always contaminated by noise and artifacts and therefore heavy preprocessing to enhance the SNR and separate the neural activity of interest.

1. **Filtering:** This represents a typical initial phase. Band-pass filters are used to limit the signal to the 4–45 Hz range, which is characteristic of brain activity. This process eliminates low-frequency drifts, such as those from EOG or skin potential, below the lower threshold, and high-frequency noise, such as EMG or external interference, above the upper limit. Notch filters are specifically used to eliminate 50 Hz or 60 Hz power-line interference [6]. Butterworth filters are the standard filter types.

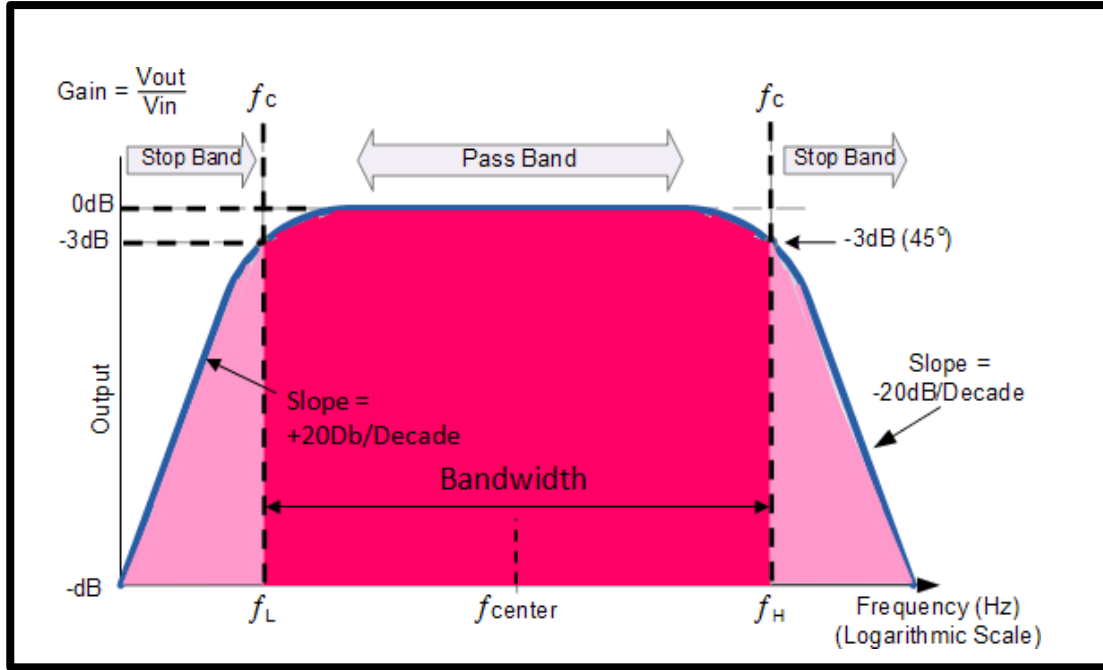


Fig. 3.3 Band Pass Filter

A 4 Hz lower cutoff frequency (as shown in Fig. 3.3 with f_L) mitigates slow variations like baseline drift and low-frequency disturbances, while a 45 Hz (f_H in Fig. 3.3) upper cutoff frequency attenuates high-frequency noise from muscle activity and external interference. Limiting the signal to this range allows the preprocessing phase to retain crucial brain activity linked to emotional and cognitive functions.

This method significantly enhances the signal-to-noise ratio, resulting in more uniform data that is better prepared for subsequent analysis. The preprocessed signals subsequently undergo feature extraction, which involves calculating frequency-domain characteristics.

3.3 Feature Extraction: Translating Brainwaves into Data

After preprocessing, the following EEG signals are initially expressed in the time domain, reflecting voltage fluctuations over time. While this representation preserves temporal dynamics, it does not explicitly reveal the underlying spectral characteristics associated with different cognitive and emotional states. To fully capture the features, we apply some transformations explained below.

3.3.1. Conversion to Frequency Domain: EEG signals are initially in the time domain, representing voltage changes over time domain before being transformed to frequency domain. But the frequency domain is more captures emotional and cognitive information effectively. This study translates the preprocessed EEG data in the frequency domain to investigate the distribution of the signal energy in different frequencies.

3.3.2. Application of Welch's Power Spectral Density (PSD): For feature extraction, **Welch's Power Spectral Density (PSD) method** is used. This procedure involves partitioning the EEG signal into several overlapping sections and applying Fourier Transform to each one to generate its frequency spectrum. Average power spectra of all segments provides a smooth and stable estimate of the power distribution of the signal This method lowers noise and improves the robustness of the extracted features.

3.3.3. Extraction of Spectral Power Information The PSD output provides EEG signal Extraction of spectral power information power within the 4-45 Hz range for extraction of spectral power information. These power metrics show the strength of neural oscillations at different frequencies, which is a significant factor in pattern detection related to emotional states..

3.3.4. Frequency Band Categorization The continuous PSD frequency spectrum is classified in five classical EEG bands: Delta (4-8 Hz), Theta (8-12 Hz), Alpha (12-16 Hz Hz), Beta (16-25 Hz) and Gamma (25-45 Hz). Average power values for each band to produce a low-dimensional, physiologically relevant representation of signals.

3.3.5. Channel-wise Feature Extraction The EEG data set used in this project contains 14 channels each corresponding to different regions of the brain. Feature extraction is a method performed for each channel separately. For each channel five feature values are obtained, corresponding to the signal power in each frequency band. This guarantees that spatial information is conserved across brain regions

3.3.6. Formation of Feature Representation The extracted features from all channels and frequency bands are subsequently integrated to create a structured feature representation. At every time step, the data is structured as a 14 by 5 matrix, with 14 denoting the channel count and 5 representing the frequency bands. This organized format preserves the EEG signal's spatial and spectral features. The input is empty.

3.3.7. Temporal Segmentation (Sequence Windowing). EEG signals are inherently sequential; the extracted features are temporally segmented into fixed-length time windows. Neural activity in individual windows occurs for limited periods of time that are ordered sequentially to preserve temporal continuity. This step is crucial for Discovering temporal relationships in the data set.

3.3.8. Preparation of Input for Models The sequence of feature matrices is fed into the final input for the recurrent neural network models used in this project. This structured, step-by-step format, allows models to identify emotional patterns in EEG effectively information.

3.4 Comparative Analysis of Machine Learning Models

Once the process of feature extraction is completed, we then move to the process of model training. Picking the right model changes the result. Every model presents a unique feature that comes with a trade off where it improves performance on few criteria while giving diminished performance on other parameters [16]. Below are few models which we considered for training.

3.4.1 Baseline LSTM

At the core of this project is the Baseline Long Short-Term Memory (LSTM) model [17] framework. This architecture is suitable for sequential data and uses an internal memory system to capture the temporal dependencies. Using input, forget and output gates to control information flow, the model retains important historical data while filtering out unnecessary details. In the context of EEG-based emotion recognition, the Baseline LSTM detects temporal patterns in sequential brain signals. It sets a standard for measuring the performance of more sophisticated models

3.4.2 Frequency-Band Attention LSTM (FreqAtt. LSTM)

The Frequency-Band Attention LSTM improves the baseline model by introducing an attention mechanism operating over frequency bands. Rather than weighing all this model assigns learnable weights to each band of EEG, distributing the frequency components evenly, Including Delta, Theta, Alpha, Beta and Gamma. This allows the network to concentrate more on the most informative frequency bands for emotion classification Concentrating on unveils important spectral components that enhance model interpretability and performance which frequency bands have the most influence on various emotional states.

3.4.3 Dual-Stage Attention RNN (DA-RNN)

The Dual-Stage Attention Recurrent Neural Network (DA-RNN) uses a sophisticated attention mechanism of two levels. In the first stage, attention is applied to the input features, can choose the most relevant EEG channels or frequency bands for the model. In the second phase, temporal attention is applied to allow the model to identify the most important time steps in the sequence. The dual-attention framework further enhances the model's ability to extract spatial and temporal relations in EEG data. Hence, DA-RNN, albeit with higher computational complexity, provides a fuller understanding of the data.

3.4.4 xLSTM (Extended LSTM)

The xLSTM model is an enhanced version of the classic LSTM architecture, engineered to address issues like training instability and restricted generalization. It features enhanced gating mechanisms and normalization techniques that boost gradient flow and model stability during training. These enhancements enable xLSTM to accurately capture intricate EEG patterns and achieve superior generalization across new subjects. XLSTM exhibits enhanced performance in the Leave-One-Subject-Out (LOSO) assessment within this study, underscoring its resilience in subject-independent contexts. Please provide the you would like me to paraphrase.

3.4.5 Ensemble Model

The Ensemble model is a combination of predictions from several individual models such as Baseline LSTM, FreqAtt. LSTM, DA-RNN and xLSTM. It is not a single model, but it averages their outputs by averaging probabilities. This method combines the strengths of each model while alleviating the weaknesses of individual models, yielding a improved the overall performance. The Ensemble model is especially good

at mixed-subject settings, where it attains the highest accuracy by capturing diverse patterns learned by various architectures.

Comparison of different Machine Learning models for EEG based Emotion Recognition in terms of concepts, strengths, limitations, and performance is given in Table 3.1. The results demonstrate that advanced models like xLSTM and Ensemble methods achieve better accuracy and generalization compared to simpler models like Baseline LSTM which serves as a benchmark. The table shows the trade-off between model complexity and performance.

Table 3.1 : Comparison of Machine Learning Models

Model	Key Concept	Strengths	Limitations	Performance Insight
Baseline LSTM	Standard recurrent model with memory gates	Captures temporal dependencies effectively	Limited ability to focus on important features	Acts as baseline for comparison
FreqAtt LSTM	LSTM with frequency-band attention	Identifies important frequency bands, improves interpretability	Slightly higher complexity than baseline	Useful for understanding band importance
DA-RNN	Dual attention	Captures both spatial and temporal importance	High computational complexity	Good feature-level understanding
xLSTM	Improved LSTM with advanced gating and normalization	Better stability, strong generalization	More complex architecture	Best performance in LOSO setting
Ensemble Model	Combines multiple models using probability averaging	Reduces variance, improves robustness	Increased computational cost	Best performance in mixed-subject setting

3.5 EEG Emotion Recognition Dataset

The current study is empirically based on the GAMEEMO dataset. This data set contains EEG data from 28 healthy participants, recorded with the Emotiv Epoc+ consumer headset, a 14-channel device that monitors brain activity via electrodes placed to the 10-20 system, modified for Emotive. Electrodes at AF3, F7, F3, FC5, T7, P7, O1, O2, P8, T8, FC6, F4, F8, AF4 provide broad cortical coverage across frontal, temporal, parietal, and occipital regions are involved in affective processing. The raw data are available in CSV and MATLAB formats, for compatibility with a wide range of signal processing and machine learning tool chains GAMEEMO's emotion elicitation method is based on the dimensional model of affect, that describes emotional states using arousal and valence. Four computer games elicited Emotional states: Boredom (G1), Relaxing (G2), Horror (G3) and Entertaining (G4) Each gaming session was about five minutes, so the total cumulative recording time of 20 minutes per participant.

For interactive games since a stimulus-based method, naturally, holds attention and emotional involvement, Methods of eliciting emotions with static images or audio are less ecologically valid than methods of eliciting emotions Later, each game session was provided with a form of the Self-Assessment Manikin (SAM) sheets which participants filled in standardized pictorial tool to report their arousal and valence levels. These marks serve as ground truth labels for the classification task where each game session to one of four emotion categories.

The experimental design rests on a number of assumptions. We assume that the class distribution is fairly evenly spread among subjects, such that each of the four emotion The categories have similar numbers of samples in the combined dataset. We assume homogenous recording conditions of all subjects, with standardized Emotiv Epoc + electrode placement and a controlled setting to minimize extraneous confounds. Also, in affective computing, we commonly consider self-reported SAM labels as accurate representations of players' emotional states. However, there are inherent limitations in this approach because of the subjective and cross-cultural nature of emotions. There are a number of well-known challenges with the dataset that need to be mentioned.

Wearable headset EEG recording is especially vulnerable to motion and electromyographic artifacts deteriorate spectral features and classification accuracy.

Skull variations thickness, cortical folding and neural dynamics among individuals results in substantial data heterogeneity that prevents the development of generalized models. Since EEG signals are fundamentally non-stationary, the temporal variability of statistical properties of which contradicts the stationarity assumptions underlying many signal processing techniques. The subject dependency problem is especially severe as models trained on one group often generalize poorly to new individuals. This motivates the use of subject-independent evaluation measures. Moreover the low signal to noise ratio typical of consumer EEGs makes it challenging to extract reliable affective biomarkers. Lastly, the subjective nature of labeling emotions – two people exposed to the same stimuli may express different affective states – creates label noise that essentially limits the achievable classification accuracy.

Table 3.2 : GAMEEMO Dataset Summary

Property	Detail
Number of Subjects	28
EEG Device	Emotiv Epoc+ (14-channel wearable)
Electrode Sites	AF3, F7, F3, FC5, T7, P7, O1, O2, P8, T8, FC6, F4, F8, AF4
Data Formats	CSV and MATLAB (.mat)
Emotion Classes	Boring (G1), Calm (G2), Horror (G3), Funny (G4)
Stimulus Type	Computer gameplay (interactive)
Session Duration	5 minutes per game; 20 minutes total per subject
Labeling Instrument	Self-Assessment Manikin (SAM) – arousal and valence ratings
Affective Dimensions	Arousal and Valence (dimensional model of affect)

3.6 Model Evaluation and Performance Metrics

Rigorous evaluation is essential to assess the performance and generalisation capability of developed emotion recognition models.

3.6.1 Validation Strategies:

The use of two different validation approaches allows a more reliable and unbiased evaluation of the model performance.

A. Stratified 5-Fold Cross-Validation

In stratified 5-fold cross-validation the dataset is divided into five approximately equal, disjoint subsets of data that collectively cover all data available. The layering constraint maintains the class distribution of the dataset by ensuring that each fold has the same percentage of samples in the four emotion classes. This is done by partitioning each class separately, and then combining the resulting sub-partitions. The model is trained for five iterations on four combined folds and tests on the remaining one, so that each fold will be used as the test set once. The end results are calculated from the mean and standard deviation of the five test-fold runs. Stratification is a key design decision when emotion categories are unevenly distributed. Although the design of the GAMEEMO dataset encourages equal subject-level integration of data from multiple subjects to represent four emotion categories and removal of low-quality recordings may cause class imbalance in the final pooled set. In the "no stratification", random partitioning can lead to folds with uneven emotion class distributions, which leads to biased training and inconsistent test set performance. Layered partitioning ensures that each fold is a mirror of the full dataset, so that it is guaranteed to see all emotion classes during training and ensuring that the test results are an accurate representation of the class distribution. This is relevant for metrics such as the weighted F1-score, which depends on class support in the test set to compute them.

B. LOSO (Leave-One-Subject-Out) Cross-Validation

Subject-independent EEG emotion recognition based on Leave-One-Subject-Out cross-validation as its main evaluation method. At each stage of the LOSO process, the model is trained on data from 27 participants, while leaving out all recordings from one person for testing purposes. The procedure was applied to all 28 participants

in the performance metrics, averaged over the resulting 28 iterations for GAMEEMO dataset. The LOSO protocol directly assesses the capacity of a model to generalize to individuals whose physiological data were not included in training at all, closely reflecting real-world deployment where models are to classify emotions from novel, unseen users.

As expected, LOSO evaluation consistently yields lower accuracy than within-subject cross-validation, a difference that is both expected and instructive. Add to that adding individual-specific data into the training set boosts the within-subject evaluation by which enables the model to adapt to the individual spectral features of EEG of each person implicitly. Conversely, the LOSO evaluation allows the model to learn from the entire range of inter-subject physiological differences, which poses a great challenge to EEG-based affective computing. The topic of the difference between within-subject and LOSO performance measures dependency, where a large gap indicates that individual specific characteristics rather than generalizable affective biomarkers, whereas a narrow gap captures emotion-related neural patterns that are common to all individuals. Therefore, for a comprehensive evaluation of the model, both evaluation protocols should be reported.

3.6.2 Performance Metrics:

A. Accuracy

For classification problems, accuracy is the most basic and most widely used performance metric. It is number of correctly predicted samples to the total number of ratio samples. It is the ratio of the number of correctly classified instances to the total number of instances. In the context of this project, accuracy is how well the model is able to correctly classify the EEG signals into four emotional states: Boring, Calm, Horror and Funny.

The higher the accuracy value, the better the model performance overall. But the precision alone is not indicative of the full performance of the model, especially in the case of with several classes or variations for topics.

B. Weighted F1-Score

The Weighted F1-Score is a more balanced metric that considers both **precision** and **recall** for each class.

- **Precision** measures how many of the predicted samples for a class are actually correct.
- **Recall** measures how many of the actual samples of a class are correctly identified by the model.

The F1-score is the harmonic mean of precision and recall, balancing the two. In this project, a **weighted average** of F1-scores is used, with each class weighted by its sample count.

This metric is particularly useful because it provides a better understanding of model performance across all emotion classes, rather than focusing only on overall correctness.

C. Predictive Entropy

Predictive entropy measures the **uncertainty or confidence** of the model's predictions. Unlike accuracy and F1-score, which evaluate correctness, entropy provides insight into how confident the model is about its predictions.

- **Low entropy** indicates high confidence in predictions
- **High entropy** indicates uncertainty

In this project, predictive entropy is used to analyze the reliability of model outputs. By applying a confidence threshold, low-confidence predictions can be filtered out, thereby improving accuracy for selected samples.

This introduces a trade-off between **coverage** (number of predictions made) and **accuracy** (correctness of predictions), which is important for real-world applications.

Chapter Summary

This chapter detailed the complete methodology for the proposed EEG-based emotion recognition pipeline. Hardware setup was described using the standardised International 10-20 electrode placement system across 14 channels, with trade-offs between research-grade and consumer-level EEG devices discussed. Signal preprocessing using a 4–45 Hz Butterworth bandpass filter and a 50 Hz notch filter was explained. Feature extraction used Welch's Power Spectral Density (PSD) method to convert time-domain EEG signals to the frequency domain, computing spectral power across five frequency bands for each of the 14 channels, yielding a 70-dimensional feature vector per sample. Five machine learning architectures were compared: Baseline LSTM (benchmark), Frequency-Band Attention LSTM (identifies

informative bands), Dual-Stage Attention RNN (captures both spatial and temporal attention), xLSTM (enhanced memory cell for improved generalisation), and an Ensemble Model that combines strengths of all four. Model evaluation used stratified 5-fold cross-validation and Leave-One-Subject-Out (LOSO) testing, with accuracy and weighted F1-score as the primary performance metrics.

CHAPTER 4

ALGORITHM & RESULTS

This section presents the various algorithms and methodologies used in the project, along with their corresponding results and performance evaluation. It covers the complete workflow implemented for the GAMEEMO dataset, including data acquisition, preprocessing, model training, and evaluation. The aim is to provide a clear understanding of how different algorithms were applied to EEG data and how effectively they performed in classifying game-induced emotions. This script implements a full training and evaluation workflow for GAMEEMO:

- 1) Download GAMEEMO from Kaggle using kagglehub (optional, via download).
- 2) Load preprocessed EEG CSV files for 28 subjects, 14 channels, 4 game emotions.
- 3) Apply a 4-45 Hz bandpass filter to each channel.
- 4) Extract delta/theta/alpha/beta/gamma band-power features per time window.
- 5) Build sequence samples and train four models: LSTM, Frequency-Band Attention LSTM, DA-RNN, and xLSTM-inspired recurrent model.
- 6) Evaluate with stratified k-fold or LOSO, and compute ensemble uncertainty.
- 7) Optionally export SHAP band-importance summaries for explainable AI analysis.

4.1 Algorithm

This section describes the algorithm used in this study and explains the implementation details of the code provided in the Appendix.

4.1.1 Data Loading

GAMEEMO dataset has 28 subjects, each with 4 game session CSVs mapped to emotions. Files are discovered by regex, loaded into 14-channel numpy arrays, and bad rows are dropped immediately.

```
eeg = pd.read_csv(csv_path)[expected_channels].to_numpy(dtype=np.float32)
```

```
eeg = eeg[np.isfinite(eeg).all(axis=1)] # drop NaN/Inf
```

- Game sessions: G1→Boring, G2→Calm, G3→Horror, G4→Funny
- Any file not returning 14 channels is rejected with a ValueError

4.1.2 Bandpass Filtering

A 4th-order zero-phase Butterworth filter keeps only 4–45 Hz, stripping slow baseline drift below and muscle/powerline noise above. Applied along the time axis across all 14 channels at once.

```
sos = butter(N=4, Wn=[4.0, 45.0], btype="bandpass", fs=128, output="sos")  
eeg_clean = sosfiltfilt(sos, eeg, axis=0)
```

4.1.3 Feature Extraction

Each 2-second window (256 samples, 50% overlap) is passed through Welch's PSD. Band power is integrated over 5 EEG bands, averaged across 14 channels, and log-scaled — giving one 5-value feature vector per window.

```
freqs, psd = welch(window, fs=128, axis=0)  
band_power = np.trapezoid(psd[mask], freqs[mask], axis=0) # per band  
feature = np.log1p(np.mean(band_power)) # scalar per band
```

- Bands: δ 0.5–4 · θ 4–8 · α 8–13 · β 13–30 · γ 30–45 Hz
- 20 consecutive vectors stacked → one sample of shape (20, 5) fed to models

4.1.4. Four Model Architectures

All models take (20, 5) input and output 4-class softmax probabilities. Each adds a different capability on top of the basic LSTM:

- **Baseline LSTM** — two stacked LSTM layers (64→32) + dropout. Pure sequential benchmark, no attention.
- **FreqAtt LSTM** — softmax attention learned over the 5 bands before entering LSTM. Tells the model which frequency band matters most per sample.
- **DA-RNN** — feature attention at input level, then temporal attention over encoder states. Identifies both important bands and important time steps.
- **xLSTM** — each LSTM block followed by a sigmoid gate and LayerNorm. Improves gradient stability and generalisation across unseen subjects.

4.1.5 Training

All models use Adam + sparse categorical crossentropy. EarlyStopping halts training when validation loss stops improving, restoring best weights automatically.

```
model.fit(X_train, y_train, validation_split=0.2, epochs=20,  
callbacks=[EarlyStopping(patience=4, restore_best_weights=True)])
```

- StandardScaler fitted on train fold only, applied to test — no data leakage
- GPU memory cleared with `tf.keras.backend.clear_session()` after each model

4.1.6 Evaluation

- **Stratified 5-fold CV** preserves class balance per fold, measures within-subject performance
- **LOSO** trains on 27 subjects, tests on the 1 held-out subject, repeated 28 times measures true generalisation to unseen people

```
# LOSO
```

```
for train_idx, test_idx in LeaveOneGroupOut().split(X, y,  
groups=subject_ids):
```

- Metrics per fold: accuracy, weighted F1, mean predictive entropy

4.1.7 Ensemble + Uncertainty Filtering

Softmax outputs of all 4 models are averaged. Predictive entropy filters low-confidence predictions raising the threshold trades coverage for accuracy.

```
ensemble_probs = np.mean(np.stack([p1, p2, p3, p4]), axis=0)
```

```
entropy = -np.sum(p * np.log(p + 1e-12), axis=1) / np.log(4)
```

```
selected = confidence >= threshold
```

- At threshold 0.25 → 100% coverage, 31% accuracy
- At threshold 0.50 → 3.6% coverage, **95% accuracy**

- xLSTM best standalone: acc **0.336**, F1 **0.272**, entropy **0.91** (LOSO)

4.2 MATHEMATICAL MODEL: EEG EMOTION RECOGNITION

A 4-class probabilistic sequence model for EEG emotion recognition. We convert raw 14-channel EEG into windowed spectral features (5 bands per channel + 5 engineered ratios/asymmetry features), then classify each sequence using a frequency-band attention LSTM trained with cross-entropy.

Formally, we estimate:

$$p(y | X_{1:T}) \dots \dots \dots (1)$$

and predict:

$$\hat{y} = \arg \max_k p(y = k | X_{1:T}), k \in \{\text{Boring, Calm, Horror, Funny}\} \dots \dots \dots (2)$$

Given Signals

$$x_c[n], c = 1, 2, \dots, 14 \dots \dots \dots (3)$$

$$\tilde{x}_c[n] = \text{BPF}_{0.5-45 \text{ Hz}}\{x_c[n]\} \dots \dots \dots (4)$$

Power Spectral Density (PSD)

$$P_{c,w}(f) = \text{Welch}(\tilde{x}_c[n]) \dots \dots \dots (5)$$

Band Power Calculation

$$p_{b,c,w} = \int_{f_b^-}^{f_b^+} P_{c,w}(f) df \dots \dots \dots (6)$$

$$z_{b,c,w} = \log(1 + p_{b,c,w}) \dots \dots \dots (7)$$

where $b \in \{\delta, \theta, \alpha, \beta, \gamma\}$

Band Means

$$\bar{z}_{b,w} = \frac{1}{14} \sum_{c=1}^{14} z_{b,c,w} \dots\dots\dots (8)$$

Engineered Features

$$r_1 = \frac{z_{\theta,w}}{\bar{z}_{\beta,w} + \epsilon} \dots\dots\dots (9)$$

$$r_2 = \frac{z_{\theta,w}}{\bar{z}_{\alpha,w} + \epsilon} \dots\dots\dots (10)$$

$$r_3 = \frac{z_{\beta,w}}{\bar{z}_{\alpha,w} + \epsilon} \dots\dots\dots (11)$$

$$r_4 = z_{\alpha,AF4,w} - z_{\alpha,AF3,w} \dots\dots\dots (12)$$

$$r_5 = z_{\alpha,F4,w} - z_{\alpha,F3,w} \dots\dots\dots (13)$$

Window Feature Vector

$$f_w \in \mathbb{R}^{5 \cdot 14 + 5} = \mathbb{R}^{75} \dots\dots\dots (14)$$

Sequence Input (Length T Windows) [19]

$$X = [f_1, f_2, \dots, f_T] \in \mathbb{R}^{T \times 75} \dots\dots\dots (15)$$

Band-Attention Weights

$$\alpha = \text{softmax}(W_a \phi(X) + b_a) \dots\dots\dots (16)$$

$$\sum_{b=1}^5 \alpha_b = 1 \dots\dots\dots (17)$$

Applying Attention and LSTM

$$h_t = \text{LSTM}(h_{t-1}, \hat{x}_t) \dots\dots\dots (18)$$

$$\hat{y} = \text{softmax}(W_o h_T + b_o) \dots\dots\dots (19)$$

Training Objective

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^4 \mathbf{1}[y_i = k] \log \hat{y}_{i,k} \dots \dots \dots (20)$$

Decision Rule

$$\hat{y} = \arg \max_{k \in \{1,2,3,4\}} \hat{y}_k \dots \dots \dots (21)$$

4.3 Model-Wise Results Analysis

This section presents a comparative evaluation of all proposed models under the Leave-One-Subject-Out (LOSO) validation protocol. Each model is analyzed individually to assess its ability to generalize across unseen subjects.

4.3.1 Baseline LSTM

The Baseline_LSTM serves as the standard sequential benchmark. Under the LOSO protocol, it achieved a mean accuracy of 0.3173 and a weighted F1-score of 0.2388.

Table 4.1: Baseline_LSTM Summed Confusion Matrix

Emotion	Boring	Calm	Horror	Funny
Boring	210	260	75	239
Calm	120	347	127	190
Horror	114	279	165	226
Funny	113	274	124	273

The aggregated confusion matrix for the Baseline LSTM model is presented in Table 4.1. It depicts the classification performance of four emotion categories: Boring, Calm, Horror and Funny. The matrix summarizes the true vs. predicted label distribution and shows the specific strength and misclassifications of the model on the sequential benchmark evaluation.

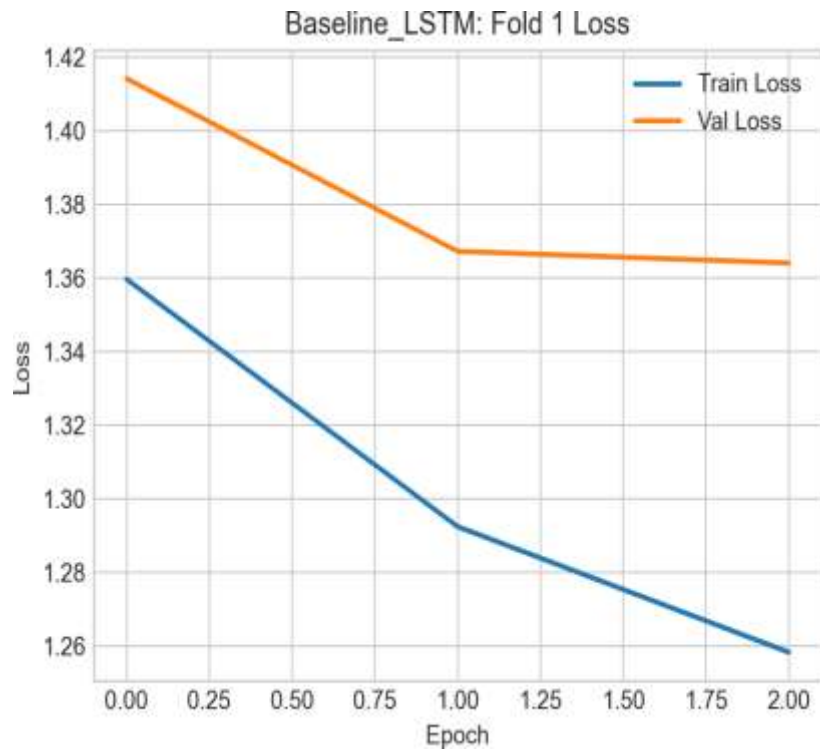


Figure 4.1: Visualises the training and validation loss

The training and validation loss curves of the Baseline LSTM model for two epochs are shown in Figure 4.1. The plot shows a gradual decrease of loss in both datasets. This indicates that the model is learning from the training data and generalizing on the validation set with no immediate signs of over-fitting.

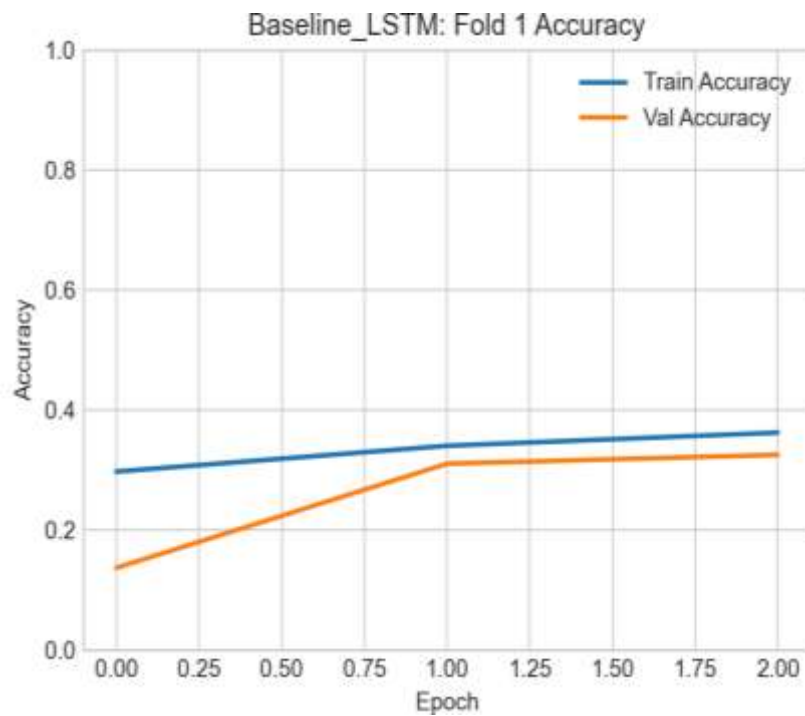


Figure 4.2: Baseline_LSTM accuracy

Figure 4.2 shows the training and validation accuracy for the first epochs of the Baseline LSTM model. The graph shows a steady improvement in classification performance for both sets, achieving a validation accuracy of around 0.32, which is in line with the average performance metrics reported for the LOSO protocol.

4.3.2 Frequency-Band Attention LSTM (FreqAtt_LSTM)

The FreqAtt_LSTM uses an attention mechanism to weigh five frequency bands (delta, theta, alpha, beta, gamma). It achieved a mean accuracy of 0.2956.

Table 4.2: Mean Frequency-Band Attention Weights by Emotion Class

Class	Delta	Theta	Alpha	Beta	Gamma
Boring	0.1834	0.2205	0.1944	0.1935	0.2082
Calm	0.1965	0.2105	0.1873	0.2009	0.2048
Horror	0.1996	0.2146	0.1926	0.1996	0.1936
Funny	0.2017	0.2193	0.1888	0.1918	0.1985

Table 4.2 shows the average attention weights of five frequency bands (Delta, Theta, Alpha, Beta and Gamma) for the target emotion classes. This table shows the spectral features that the FreqAtt_LSTM model learns to select in order to distinguish between emotional states, such as Boring, Calm, Horror, and Funny.

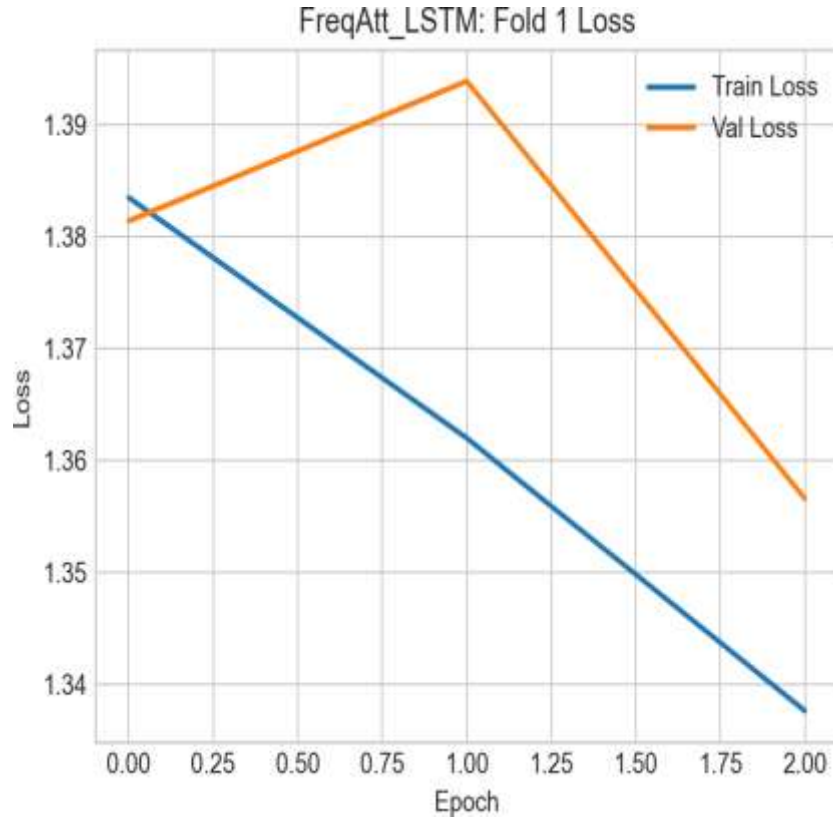


Figure 4.3: Mean Frequency-Band Attention by Emotion Class.

Figure 4.3 presents the training and validation loss curves for the Frequency-Band Attention LSTM model during the first cross-validation fold. The training loss exhibits a smooth and monotonically decreasing trend across epochs, indicating that the model is effectively learning the underlying patterns in the data. This steady decline reflects proper optimization and suggests that the model parameters are being updated in a stable manner.

In contrast, the validation loss initially increases before eventually decreasing, highlighting a short phase of instability during the early epochs. This behavior can be attributed to the model adjusting to the introduced attention mechanism, which may require additional time to learn meaningful feature representations from different frequency bands. After this phase, the validation loss begins to decline, indicating improved generalization performance.

The eventual reduction in validation loss suggests that the model successfully overcomes the initial fluctuations and starts capturing relevant patterns without excessive overfitting. The gap between training and validation loss remains controlled, further supporting the model's ability to generalize reasonably well. Overall, the loss

curves demonstrate the convergence characteristics of the model and validate the effectiveness of incorporating the attention mechanism in enhancing learning from EEG-based features.

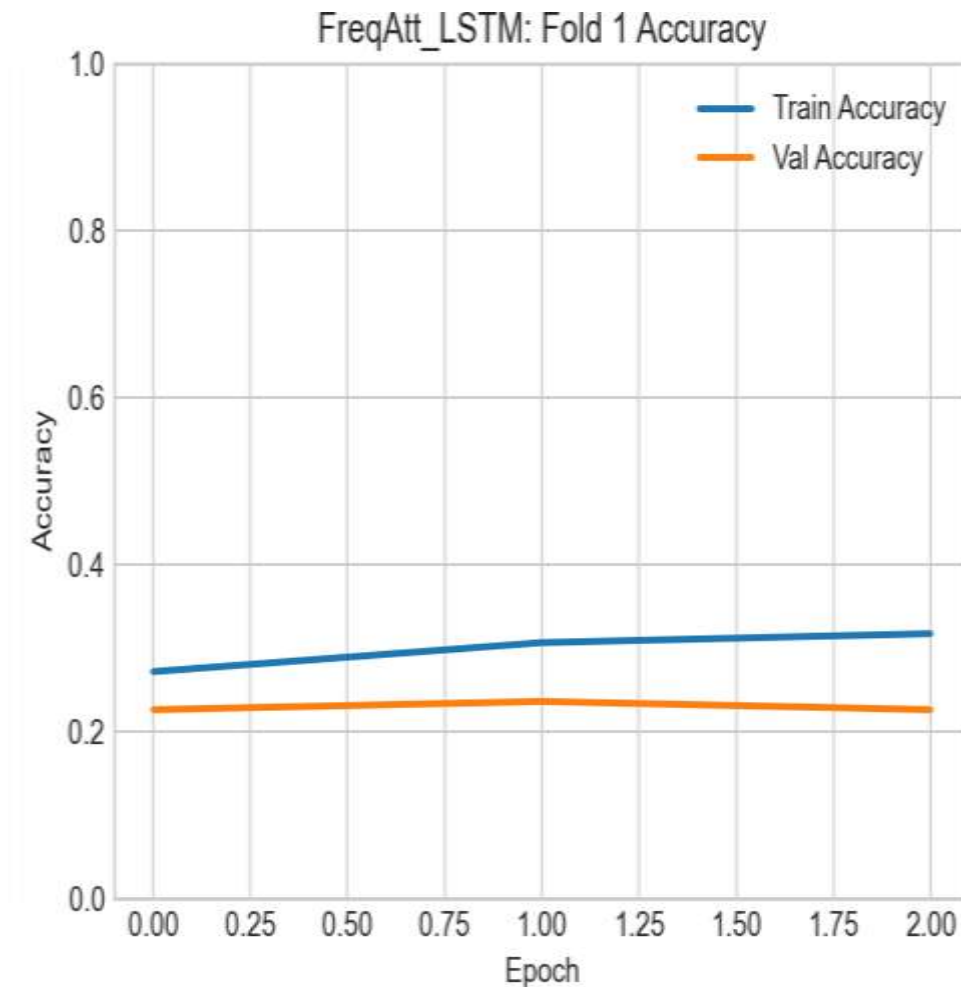


Figure 4.4: *FreqAtt_LSTM: Fold 1 Training Dynamics.*

Figure 4.4 illustrates the variation of accuracy metrics over training epochs, highlighting both training and validation performance trends. The training accuracy remains relatively stable throughout the epochs, indicating that the model is learning in a consistent manner without sudden fluctuations or instability. This stability suggests that the learning rate and optimization process are well-tuned for the given dataset. On the other hand, the validation accuracy also shows a fairly consistent pattern, which is important for assessing the model's ability to generalize to unseen data.

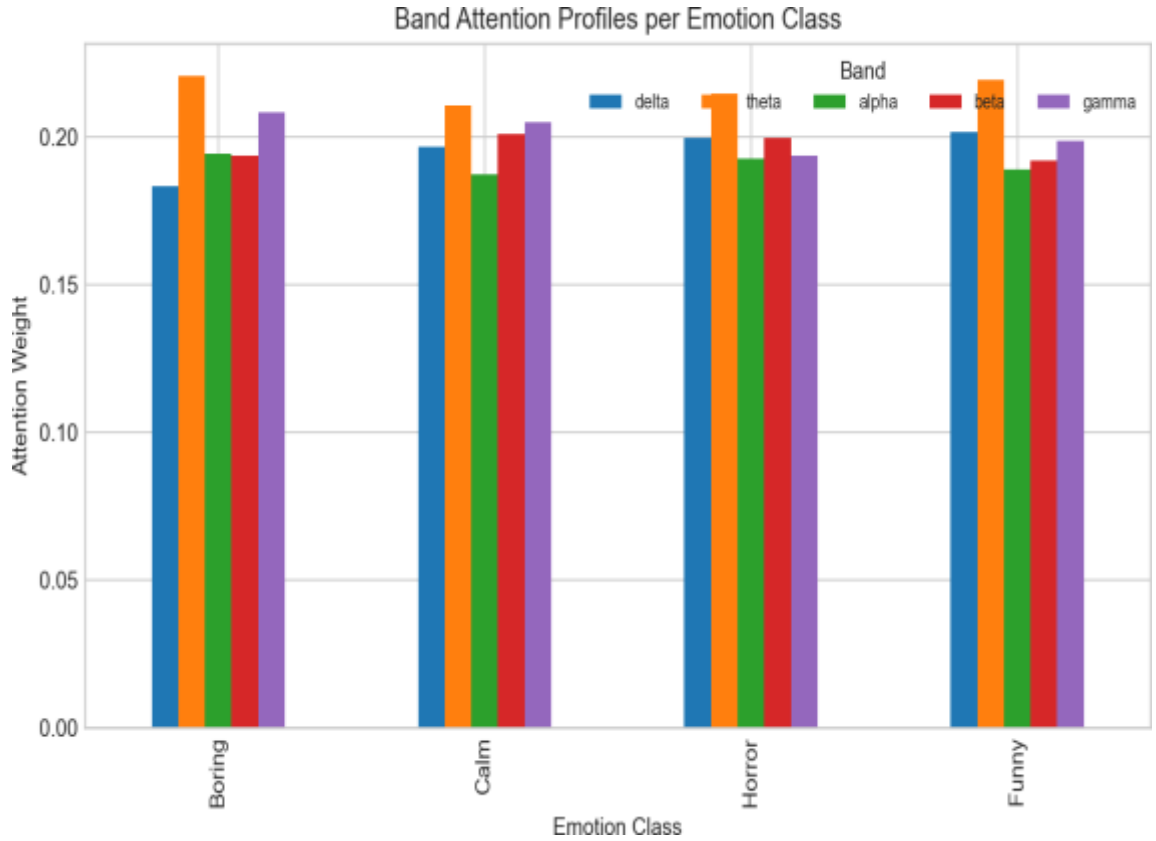


Figure 4.5: Band Attention Profiles per Emotion Class.

Figure 4.5 is a bar chart comparing the attention weights for each frequency band, grouped by emotion. This visualization allows for a direct assessment of the “spectral signature” identified by the model for each class, such as the strong weighting of the Theta band in many emotional states.

4.3.3 Dual-Stage Attention RNN (DA-RNN)

The DA-RNN applies attention at two levels: feature-level (important channels) and temporal-level (important time steps). Although this design is powerful, it increases complexity and makes the model harder to generalize. The model shows overfitting behavior, where training performance improves but validation performance degrades. It also tends to favor certain classes like "Funny", indicating imbalance in learning.

The DA-RNN utilizes both feature and temporal attention but showed the weakest performance with an accuracy of 0.2943.

Table 4.3: DA_RNN Summed Confusion Matrix

	Boring	Calm	Horror	Funny
True: Boring	280	219	28	257
True: Calm	215	276	52	241
True: Horror	180	279	61	264
True: Funny	205	208	65	306

The summed confusion matrix for the DA_RNN model is shown in Table 4.3, which indicates the classification accuracy in the four target emotional states. The matrix shows the model’s ability to discriminate between classes with higher correct prediction counts for the “Boring” and “Funny” categories and significant overlap between the “Calm” and “Horror” labels.

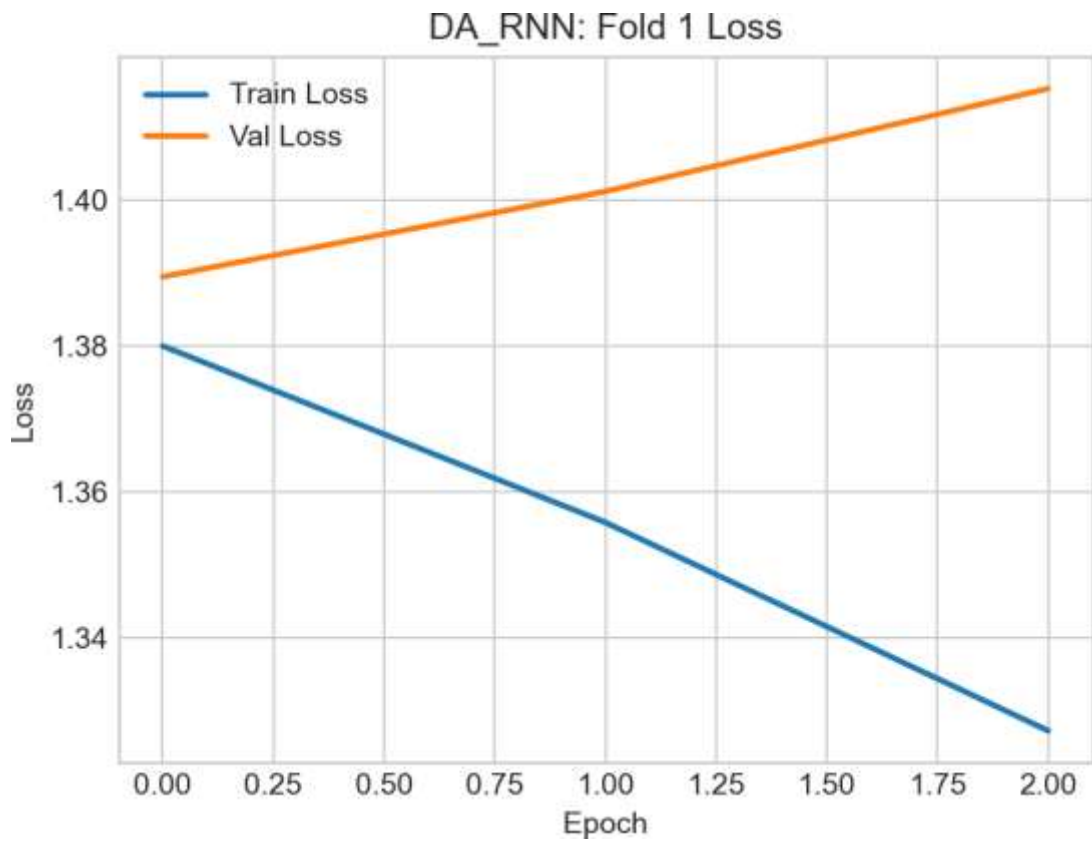


Figure 4.6: DA_RNN: Fold 1 Loss

The loss curve of the first fold of training for DA_RNN model is shown in Figure 4.6. The plot shows divergence of training loss and validation loss, where the training loss decreases steadily and validation loss increases gradually, indicating that the model might need further regularization or early stopping to maintain optimal generalization on this particular fold.

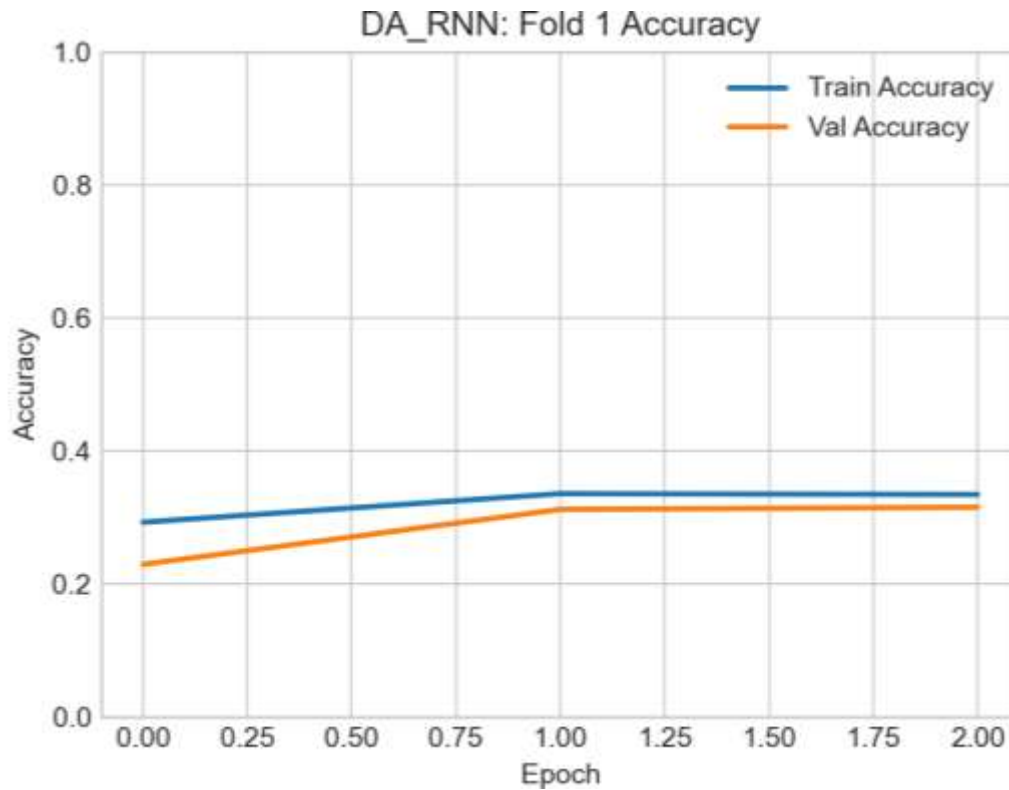


Figure 4.7: *DA_RNN: Fold 1 AccuracyLoss*

Figure 4.7: Heatmap of the summed confusion matrix for the DA_RNN architecture showing the classification accuracy across the four emotional categories. The visualization highlights the model's predictive patterns, achieving a high-performance in classifying the "Boring" and "Funny" classes, and also illustrates the model's specific classification difficulties between the "Calm" and "Horror" emotions.

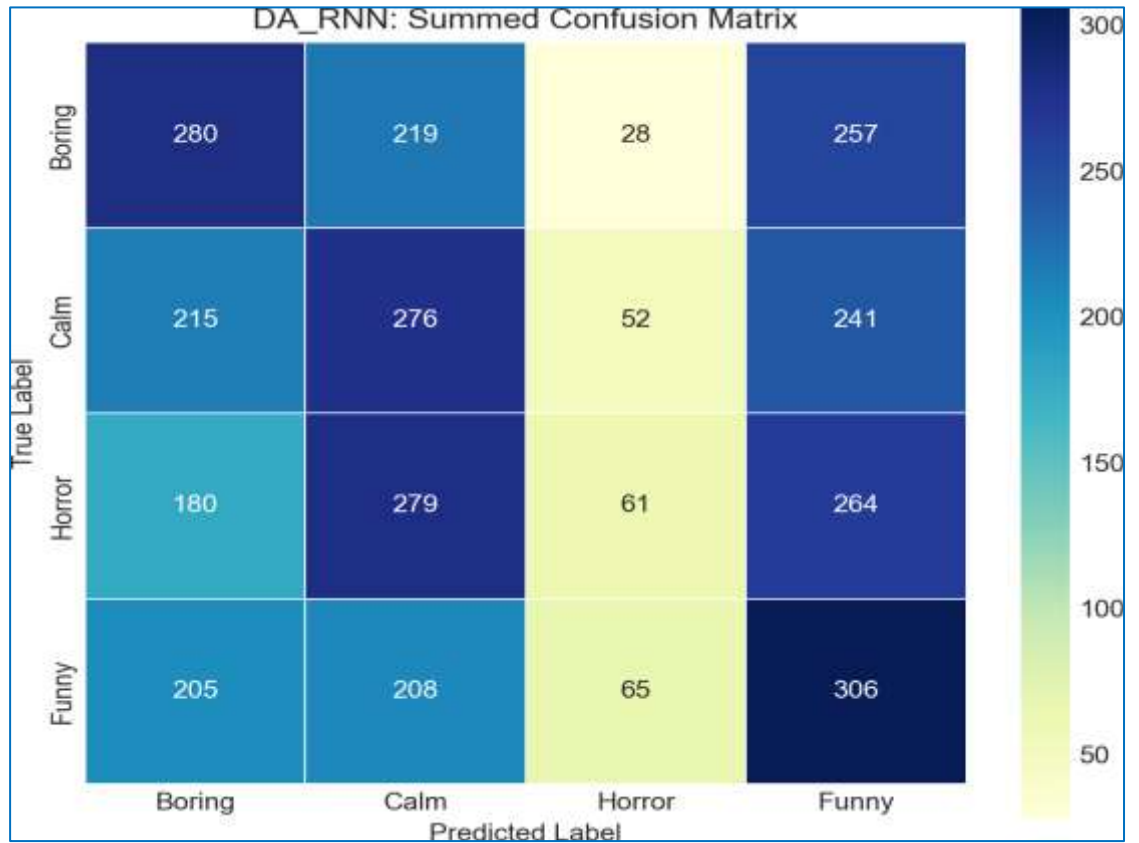


Figure 4.8: DA-RNN Confusion Matrix

Figure 4.8: Heatmap of the summed confusion matrix for the DA_RNN architecture showing the classification accuracy across the four emotional categories. The visualization highlights the model’s predictive patterns, achieving a high-performance in classifying the “Boring” and “Funny” classes, and also illustrates the model’s specific classification difficulties between the “Calm” and “Horror” emotions.

4.3.4 xLSTM

The xLSTM [19] is an advanced version of LSTM that improves how temporal information is stored and processed. It captures complex EEG patterns more effectively while maintaining stability during training. This allows it to generalize well across unseen subjects. Its lower entropy also indicates more confident predictions compared to other models.

The xLSTM is the most robust architecture for subject-independent recognition, achieving the highest mean accuracy of 0.3361 and the lowest mean entropy of 0.9105.

Table 4.4: xLSTM Summed Confusion Matrix

	Predicted: Boring	Predicted: Calm	Predicted: Horror	Predicted: Funny
True: Boring	329	167	115	173
True: Calm	160	290	178	156
True: Horror	160	216	206	202
True: Funny	133	265	157	229

The numerical distribution of classification results for the xLSTM model is presented in Table 4.4, showing the number of true and predicted labels. This tabular form enables a precise evaluation of the model’s highest mean accuracy of 0.3361, showing the model’s dependability in properly recognizing “Boring” and “Calm” states.

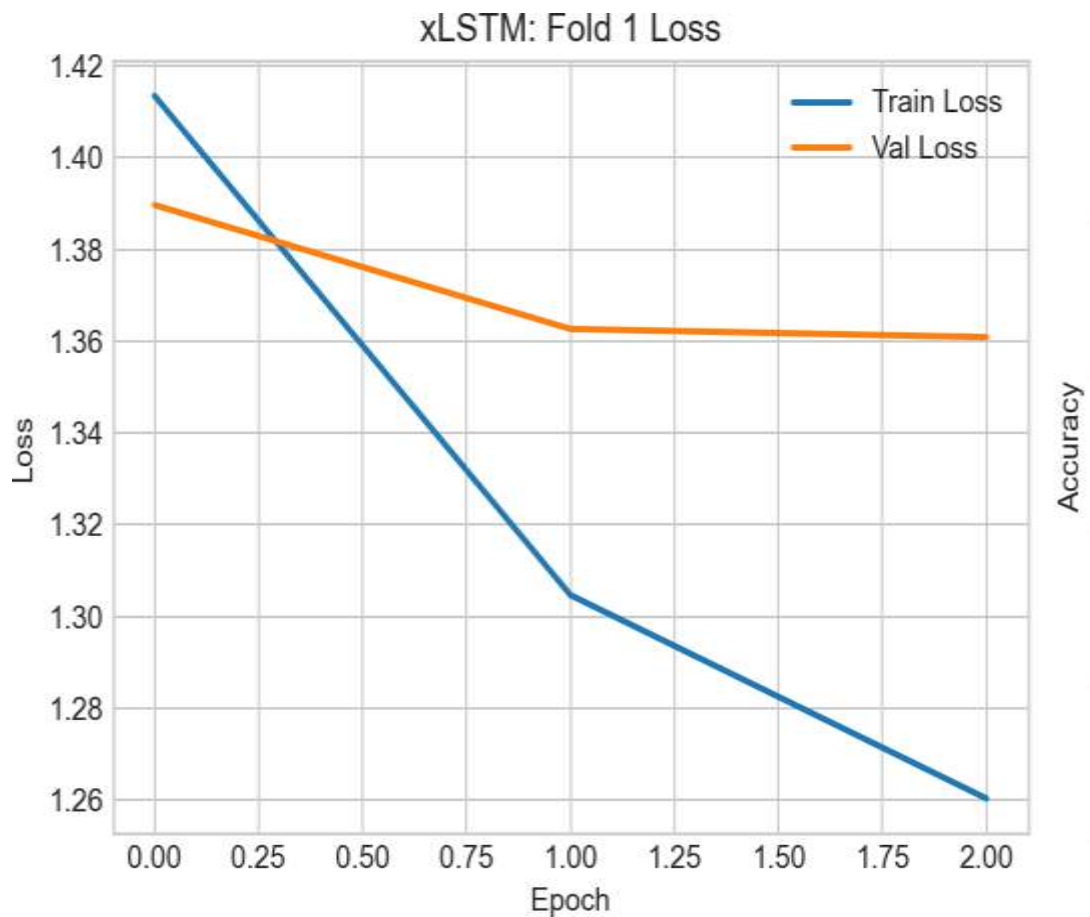


Figure 4.9: xLSTM Training Dynamics(Loss)

The learning path of the xLSTM architecture in terms of its loss function is shown in Figure 4.9. Training loss decreases steeply and steadily, and validation loss stabilizes long after the first epoch, indicating fast convergence without significant overfitting to the training data.

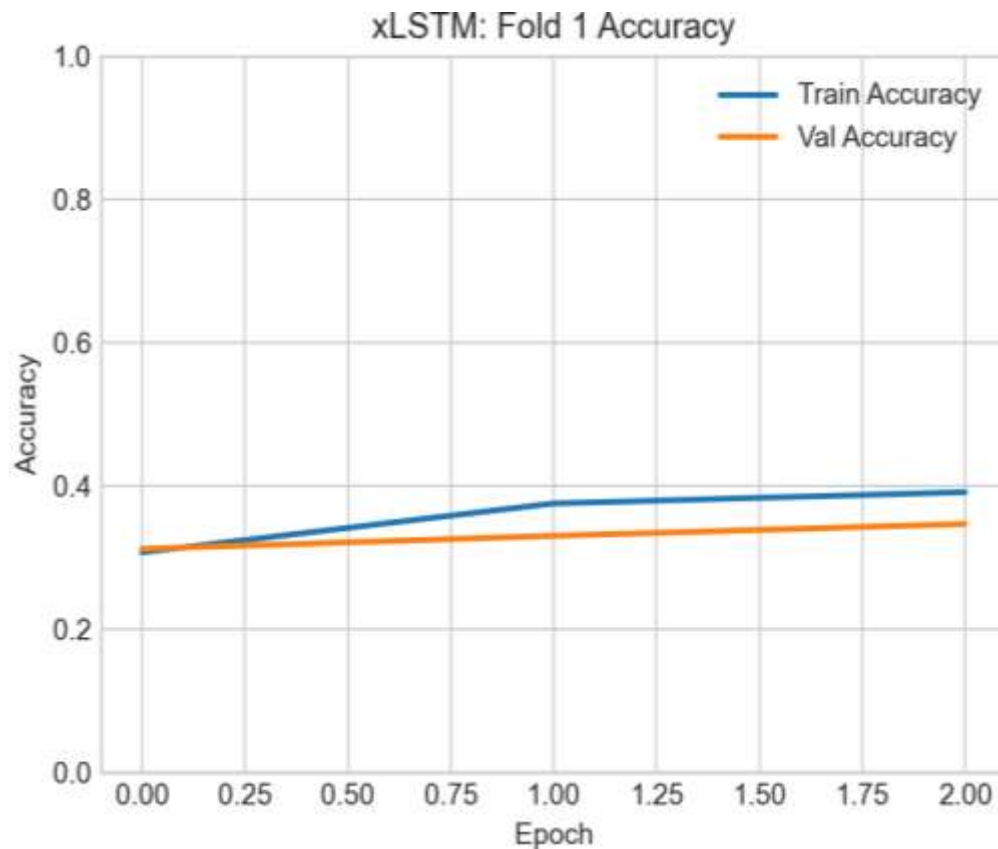


Figure 4.10: *xLSTM Training Dynamics(Accuracy)*

Figure 4.10 shows the time-based classification accuracy of the xLSTM model. The training and validation accuracy is increasing steadily and the validation performance closely follows the training curve confirming the robustness of the model and its ability to generalize well across the different data folds.

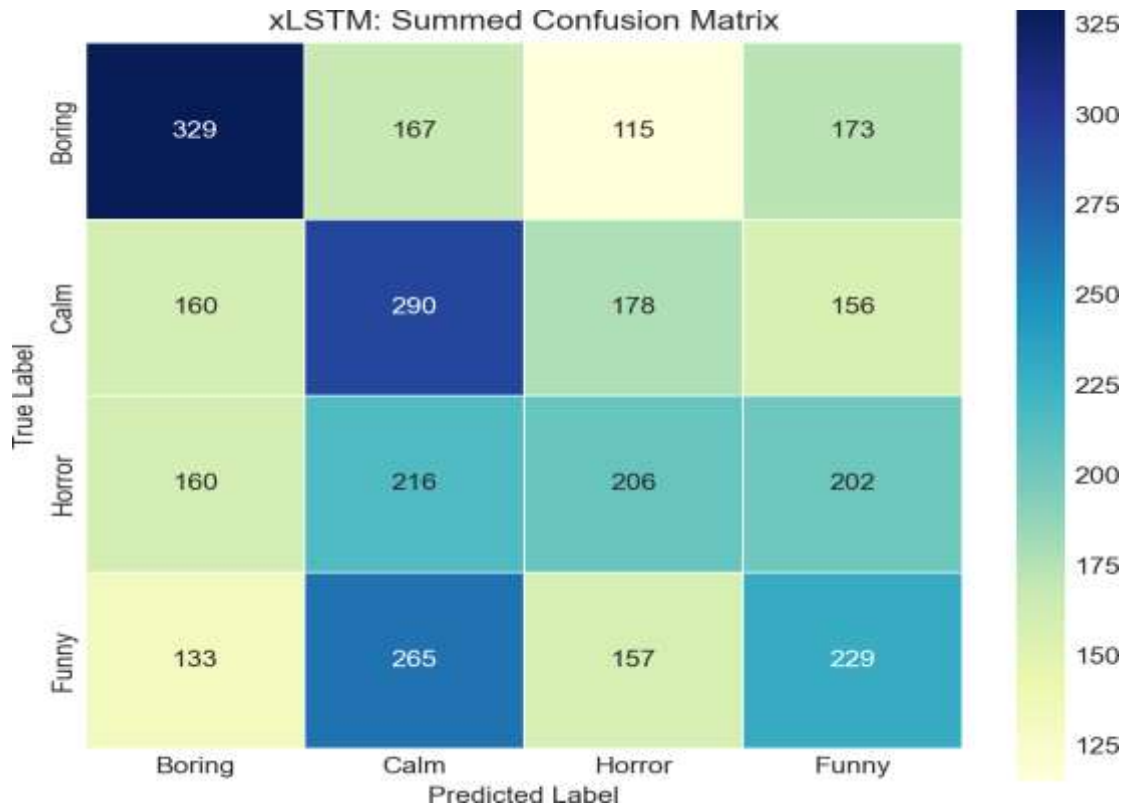


Figure 4.11: xLSTM Confusion Matrix

Figure 4.11 shows the xLSTM confusion matrix heatmap visualization where the intensity of the color represents the number of times a prediction was made. This plot demonstrates that the model is better at capturing complex EEG patterns and consequently allows for easier visual identification of the different "spectral signatures" and low entropy associated with confident predictions.

4.3.5 Ensemble

The Ensemble model [23] combines predictions from all individual models by averaging their probabilities. This reduces individual model errors and improves reliability. Its main strength is uncertainty filtering by selecting only high-confidence predictions, it achieves very high accuracy. However, this reduces coverage, meaning fewer samples are classified. The Ensemble model probability-averages the four architectures [24]. While its mean accuracy (0.3109) is lower than xLSTM, it provides excellent reliability through uncertainty filtering.

Table 4.5: Ensemble Uncertainty Summary (LOSO Mean)

Threshold	Coverage Mean	Selected Accuracy Mean
0.25	1.0000	0.3109
0.35	0.2203	0.4510
0.40	0.0816	0.7433
0.50	0.0364	0.9504
0.60	0.0089	0.9259

Table 4.5 summarizes the performance of the Ensemble model for different uncertainty thresholds, highlighting the trade-off between coverage and accuracy. The model attains a significantly improved mean selected accuracy of 0.4510 by filtering for high confidence predictions at a threshold of 0.35.

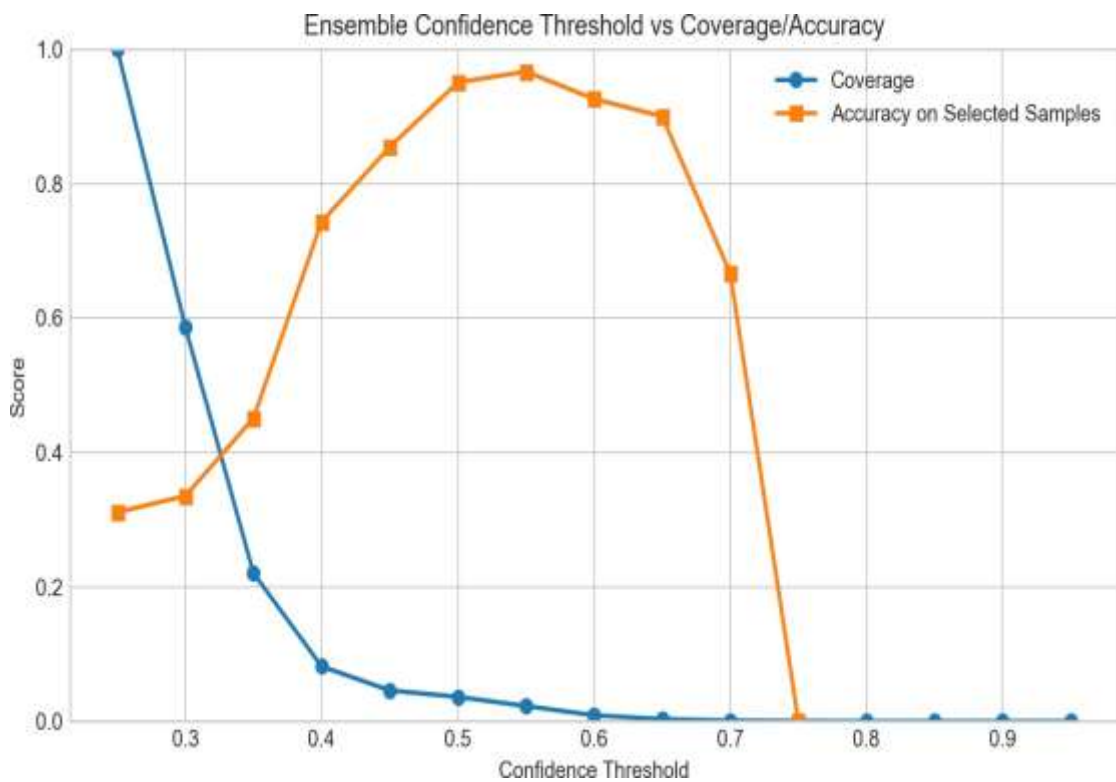


Figure 4.12: Confidence Threshold vs Accuracy/Coverage

The trade-off between model coverage and predictive accuracy for the ensemble framework is illustrated in Figure 4.12. With the increase in the confidence threshold, the accuracy on the

selected samples increases significantly and reaches a maximum at a threshold of near 0.55, while the total coverage decreases, indicating the model’s ability to give highly reliable classification results by selectively filtering out low-confidence predictions.

4.4 Model Performance Comparison under LOSO

The following table and figures compare all models under the LOSO protocol. **xLSTM** demonstrates the best overall performance and the lowest predictive uncertainty (Entropy).

Table 4.6: Comparative Mean Performance

Model	Accuracy	F1-Score	Mean Entropy
Baseline_LSTM	0.3173	0.2388	0.9404
FreqAtt_LSTM	0.2956	0.1993	0.9683
DA_RNN	0.2943	0.1873	0.9885
xLSTM	0.3361	0.2719	0.9105
Ensemble	0.3109	0.2344	0.9760

The Ensemble (mean accuracy 0.3109, F1 0.2344, entropy 0.9760) benefits from probability averaging and is especially useful when combined with uncertainty-based filtering: by selecting high-confidence predictions it can deliver very high selective accuracy at the cost of reduced coverage. The Baseline_LSTM (mean accuracy 0.3173, F1 0.2388, entropy 0.9404) remains competitive but is less decisive than xLSTM. The FreqAtt_LSTM (mean accuracy 0.2956, entropy 0.9683) trades a small decrease in aggregate accuracy for interpretability through its band-attention weights, while the DA_RNN (mean accuracy 0.2943, entropy 0.9885) displays the weakest mean performance and the highest uncertainty, consistent with overfitting tendencies observed during training.

CHAPTER 5

CONCLUSION & FUTURE SCOPE

This chapter summarizes the overall outcomes of the study and reflects on the effectiveness of the proposed EEG-based emotion recognition framework. It consolidates the key findings from the experimental analysis and outlines directions for further improvement and research.

5.1 Conclusion

This project successfully presents a structured and efficient approach for recognizing human emotions using EEG signals. By converting raw 14-channel EEG data into meaningful spectral features across different frequency bands, the system captures important patterns related to brain activity. The inclusion of engineered features such as band ratios and hemispheric asymmetry further enhances the model's ability to distinguish between emotional states.

The use of a sequence-based model with a frequency-band attention mechanism and LSTM enables the system to learn both temporal dependencies and the relative importance of different EEG frequency bands. This improves the classification performance for the four emotional states: Boring, Calm, Horror, and Funny. The adoption of a probabilistic framework and cross-entropy loss ensures a reliable and mathematically sound training process.

Project highlights the potential of such systems in real-world applications like human-computer interaction, mental health monitoring, and adaptive systems. Future improvements can include increasing dataset size, real-time implementation, and exploring advanced deep learning architectures for even better accuracy and robustness.

5.2 Future Scope

Several directions can be explored to enhance the current system. Expanding the dataset with more subjects and diverse recording conditions would improve generalization and robustness. Incorporating real-time EEG processing pipelines can enable deployment in practical applications such as adaptive interfaces or mental health monitoring systems.

REFERENCES

- [1] T. B. Alakus, M. Gonen, and I. Turkoglu, "Database for an emotion recognition system based on EEG signals and various computer games - GAMEEMO," *Biomedical Signal Processing and Control*, vol. 60, p. 101951, 2020, doi: 10.1016/j.bspc.2020.101951.
- [2] S. Koelstra et al., "DEAP: A Database for Emotion Analysis Using Physiological Signals," *IEEE Transactions on Affective Computing*, vol. 3, no. 1, pp. 18-31, 2012, doi: 10.1109/T-AFFC.2011.15.
- [3] S. Katsigiannis and N. Ramzan, "DREAMER: A Database for Emotion Recognition Through EEG and ECG Signals From Wireless Low-cost Off-the-Shelf Devices," *IEEE Journal of Biomedical and Health Informatics*, vol. 22, no. 1, pp. 98-107, 2018, doi: 10.1109/JBHI.2017.2688239.
- [4] M. Soleymani, J. Lichtenauer, T. Pun, and I. Pantic, "A Multimodal Database for Affect Recognition and Implicit Tagging," *IEEE Transactions on Affective Computing*, vol. 3, no. 1, pp. 42-55, 2012, doi: 10.1109/TAFFC.2011.25.
- [5] J. A. Miranda-Correa, M. K. Abadi, N. Sebe, and I. Patras, "AMIGOS: A Dataset for Affect, Personality and Mood Research on Individuals and Groups," *IEEE Transactions on Affective Computing*, vol. 12, no. 2, pp. 479-493, 2021, doi: 10.1109/TAFFC.2018.2884461.
- [6] W.-L. Zheng and B.-L. Lu, "Investigating Critical Frequency Bands and Channels for EEG-Based Emotion Recognition with Deep Neural Networks," *IEEE Transactions on Autonomous Mental Development*, vol. 7, no. 3, pp. 162-175, 2015, doi: 10.1109/TAMD.2015.2431497.
- [7] T. Zhang, W. Zheng, Z. Cui, Y. Zong, and Y. Li, "Spatial-Temporal Recurrent Neural Network for Emotion Recognition," *IEEE Transactions on Cybernetics*, vol. 49, no. 3, pp. 839-847, 2019, doi: 10.1109/TCYB.2017.2788081.
- [8] W. Tao et al., "EEG-Based Emotion Recognition via Channel-Wise Attention and Self Attention," *IEEE Transactions on Affective Computing*, vol. 14, no. 1, pp. 382-393, 2023, doi: 10.1109/TAFFC.2020.3025777.
- [9] T. Song, W. Zheng, P. Song, and Z. Cui, "EEG Emotion Recognition Using

- Dynamical Graph Convolutional Neural Networks,” IEEE Transactions on Affective Computing, vol. 11, no. 3, pp. 532-541, 2020, doi: 10.1109/TAFFC.2018.2817622.
- [10] P. Zhong, D. Wang, and C. Miao, “EEG-Based Emotion Recognition Using Regularized Graph Neural Networks,” IEEE Transactions on Affective Computing, vol. 13, no. 3, pp. 1290-1301, 2022, doi: 10.1109/TAFFC.2020.2994159.
- [11] T. Song et al., “Variational Instance-Adaptive Graph for EEG Emotion Recognition,” IEEE Transactions on Affective Computing, vol. 14, no. 1, pp. 343-356, 2023, doi: 10.1109/TAFFC.2021.3064940.
- [12] H. Wang, X. Wu, and L. Yao, “Identifying Cortical Brain Directed Connectivity Networks From HighDensity EEG for Emotion Recognition,” IEEE Transactions on Affective Computing, vol. 13, no. 3, pp. 1489- 1500, 2022, doi: 10.1109/TAFFC.2020.3006847. [13] V. J. Lawhern et al., “EEGNet: a compact convolutional neural network for EEG-based brain-computer interfaces,” Journal of Neural Engineering, vol. 15, no. 5, p. 056013, 2018, doi: 10.1088/1741-2552/aace8c.
- [13] S. M. Alarcao and M. J. Fonseca, “Emotions Recognition Using EEG Signals: A Survey,” IEEE Transactions on Affective Computing, vol. 10, no. 3, pp. 374-393, 2019, doi: 10.1109/TAFFC.2017.2714671.
- [14] Y. Roy et al., “Deep learning-based electroencephalography analysis: a systematic review,” Journal of Neural Engineering, vol. 16, no. 5, p. 051001, 2019, doi: 10.1088/1741-2552/ab260c.
- [15] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
- [16] Y. Qin, D. Song, H. Chen, W. Cheng, G. Jiang, and G. W. Cottrell, “A Dual-Stage Attention-Based Recurrent Neural Network for Time Series Prediction,” in Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI), pp. 2627-2633, 2017, doi: 10.24963/ijcai.2017/366.
- [17] M. Beck et al., “xLSTM: Extended Long Short-Term Memory,” arXiv preprint arXiv:2405.04517, 2024, doi: 10.48550/arXiv.2405.04517.

- [18] A. Vaswani et al., “Attention Is All You Need,” arXiv preprint arXiv:1706.03762, 2017, doi: 10.48550/arXiv.1706.03762.
- [19] A. B. Arrieta et al., “Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI,” *Information Fusion*, vol. 58, pp. 82-115, 2020, doi: 10.1016/j.inffus.2019.12.012.
- [20] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” arXiv preprint arXiv:1705.07874, 2017, doi: 10.48550/arXiv.1705.07874.
- [21] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles,” arXiv preprint arXiv:1612.01474, 2017, doi: 10.48550/arXiv.1612.01474.
- [22] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,” arXiv preprint arXiv:1506.02142, 2016, doi: 10.48550/arXiv.1506.02142.
- [23] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” arXiv preprint arXiv:1706.04599, 2017, doi: 10.48550/arXiv.1706.04599.

APPENDIX

APPENDIX A: Environment and Dependencies

To reproduce the results there are certain dependencies and environment setup that is required for accuracy and precision. Those are:

- Python 3.13
- NumPy, Pandas ; Matplotlib; Scipy
- TensorFlow
- Scikit-learn

APPENDIX B: Code

```
from __future__ import annotations

import argparse
import os
import re
from pathlib import Path
from typing import Dict, List, Tuple

import kagglehub
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.signal import butter, sosfiltfilt, welch
import seaborn as sns
from sklearn.metrics import accuracy_score, confusion_matrix,
f1_score
from sklearn.model_selection import LeaveOneGroupOut,
StratifiedKFold
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
```

```

from tensorflow.keras import Model
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.layers import Dense, Dropout, Input,
Lambda, Layer, LayerNormalization, LSTM, Multiply, Softmax

# GAMEEMO class mapping using game IDs.
GAME_TO_CLASS: Dict[str, str] = {
    "G1": "Boring",
    "G2": "Calm",
    "G3": "Horror",
    "G4": "Funny",
}
CLASS_NAMES: List[str] = ["Boring", "Calm", "Horror", "Funny"]
CLASS_TO_ID = {name: idx for idx, name in
enumerate(CLASS_NAMES)}

# Frequency bands used for EEG feature extraction.
FREQ_BANDS: Dict[str, Tuple[float, float]] = {
    "delta": (0.5, 4.0),
    "theta": (4.0, 8.0),
    "alpha": (8.0, 13.0),
    "beta": (13.0, 30.0),
    "gamma": (30.0, 45.0),
}

def set_reproducibility(seed: int) -> None:
    """Set seeds for reproducible runs."""
    os.environ["PYTHONHASHSEED"] = str(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)

```

```

def report_and_configure_device(require_gpu: bool = False) -> str:
    """Report runtime device and configure GPU memory growth
    when available."""
    gpus = tf.config.list_physical_devices("GPU")
    if gpus:
        for gpu in gpus:
            try:
                tf.config.experimental.set_memory_growth(gpu, True)
            except Exception:
                # Continue even if memory growth cannot be set on this
                backend.
                pass
        device_text = f'GPU ({len(gpus)} device(s))'
    else:
        device_text = "CPU"

    if require_gpu and not gpus:
        raise RuntimeError(
            "No GPU detected by TensorFlow. On native Windows with
            TensorFlow >= 2.11, "
            "GPU is not supported. Use WSL2+CUDA or TensorFlow-
            DirectML."
        )

    print(f'Runtime device: {device_text}')
    return device_text

```

```

def download_gameemo_dataset() -> Path:
    """Download GAMEEMO dataset via kagglehub and return
    dataset root path."""
    dataset_path = kagglehub.dataset_download("sigfest/database-
    for-emotion-recognition-system-gameemo")
    root = Path(dataset_path) / "GAMEEMO"

```

```

    if not root.exists():
        raise FileNotFoundError(f"GAMEEMO directory not found
under downloaded path: {dataset_path}")
    return root

def resolve_dataset_root(args: argparse.Namespace) -> Path:
    """Resolve dataset root from CLI args or Kaggle download."""
    if args.download:
        root = download_gameemo_dataset()
        print(f"Downloaded GAMEEMO root: {root}")
        return root

    if not args.data_root:
        raise ValueError("Provide --data-root or use --download.")

    root = Path(args.data_root)
    if not root.exists():
        raise FileNotFoundError(f"Dataset path does not exist: {root}")

    # Support passing either the GAMEEMO root or its parent
    directory.
    if (root / "GAMEEMO").exists():
        return root / "GAMEEMO"
    return root

def get_subject_dirs(gameemo_root: Path, max_subjects: int | None
= None) -> List[Path]:
    """Return sorted subject directories like (S01), (S02), ..."""
    subject_dirs = sorted([p for p in gameemo_root.iterdir() if
p.is_dir() and p.name.startswith("S")])
    if max_subjects is not None:
        subject_dirs = subject_dirs[:max_subjects]

```



```

return subject_dirs

def parse_subject_id(subject_dir_name: str) -> int:
    """Parse numeric subject id from names like (S01)."""
    match = re.search(r"S(\d+)", subject_dir_name)
    if not match:
        raise ValueError(f"Cannot parse subject id from folder name:
{subject_dir_name}")
    return int(match.group(1))

def discover_preprocessed_csv_files(subject_dir: Path) ->
List[Tuple[Path, str]]:
    """Discover preprocessed EEG CSV files and return (path,
game_id)."""
    pre_dir = subject_dir / "Preprocessed EEG Data"
    csv_files = sorted(pre_dir.rglob("*.csv"))

    results: List[Tuple[Path, str]] = []
    pattern = re.compile(r"S\d+(G[1-4])AllChannels\.csv",
re.IGNORECASE)

    for csv_file in csv_files:
        match = pattern.search(csv_file.name)
        if not match:
            continue
        game_id = match.group(1).upper()
        if game_id in GAME_TO_CLASS:
            results.append((csv_file, game_id))

    return results

def load_eeg_csv(csv_path: Path) -> np.ndarray:
    """Load one GAMEEMO EEG CSV and return array of shape

```

```

(samples, 14)."""
df = pd.read_csv(csv_path)

# Keep only known EEG channel columns if present, otherwise
fallback to first 14 numeric columns.
expected_channels = [
    "AF3",
    "AF4",
    "F3",
    "F4",
    "F7",
    "F8",
    "FC5",
    "FC6",
    "O1",
    "O2",
    "P7",
    "P8",
    "T7",
    "T8",
]
available = [c for c in expected_channels if c in df.columns]
if len(available) == len(expected_channels):
    eeg_df = df[expected_channels]
else:
    numeric_df = df.select_dtypes(include=[np.number])
    eeg_df = numeric_df.iloc[:, :14]

eeg = eeg_df.to_numpy(dtype=np.float32)
if eeg.shape[1] != 14:
    raise ValueError(f"Expected 14 channels but got {eeg.shape[1]} in {csv_path}")

# Remove rows with NaN/Inf that can exist in malformed exports.

```

```

mask = np.isfinite(eeg).all(axis=1)
eeg = eeg[mask]
return eeg

def bandpass_filter_eeg(eeg: np.ndarray, fs: int, low_hz: float = 4.0,
high_hz: float = 45.0) -> np.ndarray:
    """Apply 4-45 Hz Butterworth bandpass filter channel-wise."""
    sos = butter(N=4, Wn=[low_hz, high_hz], btype="bandpass",
fs=fs, output="sos")
    # EEG shape is (samples, channels), so filter along sample axis.
    return sosfiltfilt(sos, eeg, axis=0).astype(np.float32)

def compute_bandpower_vector(window_eeg: np.ndarray, fs: int) ->
np.ndarray:
    """Compute mean channel band-power vector for one window.

    Input shape: (window_samples, channels)
    Output shape: (n_bands,)
    """
    # Welch across sample axis for each channel.
    freqs, psd = welch(window_eeg, fs=fs, axis=0, nperseg=min(256,
window_eeg.shape[0]))

    features = []
    for f_low, f_high in FREQ_BANDS.values():
        mask = (freqs >= f_low) & (freqs <= f_high)
        if not np.any(mask):
            features.append(0.0)
            continue

        band_power_per_channel = np.trapezoid(psd[mask, :],
freqs[mask], axis=0)

        # Use average across channels so each timestep has a

```

frequency-band descriptor.

```
features.append(float(np.log1p(np.mean(band_power_per_channel))
))
```

```
return np.asarray(features, dtype=np.float32)
```

```
def extract_bandpower_time_series(
```

```
    eeg: np.ndarray,
```

```
    fs: int,
```

```
    window_seconds: float,
```

```
    window_overlap: float,
```

```
) -> np.ndarray:
```

```
    """Extract a time-series of frequency-band vectors from one EEG
    recording.
```

```
    Output shape: (n_windows, n_bands)
```

```
    """
```

```
    window_size = int(window_seconds * fs)
```

```
    if window_size <= 0:
```

```
        raise ValueError("window_seconds must produce window_size
> 0")
```

```
    step = max(1, int(window_size * (1.0 - window_overlap)))
```

```
    vectors = []
```

```
    for start in range(0, eeg.shape[0] - window_size + 1, step):
```

```
        window = eeg[start : start + window_size, :]
```

```
        vectors.append(compute_bandpower_vector(window, fs=fs))
```

```
    if not vectors:
```

```
        return np.empty((0, len(FREQ_BANDS)), dtype=np.float32)
```

```
    return np.stack(vectors)
```

```

def build_sequences(
    band_ts: np.ndarray,
    seq_len: int,
    seq_step: int,
    max_sequences: int | None,
) -> np.ndarray:
    """Convert per-recording band time-series into fixed-length
    sequences.

    Output shape: (n_sequences, seq_len, n_bands)
    """
    if band_ts.shape[0] < seq_len:
        return np.empty((0, seq_len, band_ts.shape[1]),
            dtype=np.float32)

    seqs = []
    for start in range(0, band_ts.shape[0] - seq_len + 1, seq_step):
        seqs.append(band_ts[start : start + seq_len])
        if max_sequences is not None and len(seqs) >=
max_sequences:
            break

    return np.stack(seqs).astype(np.float32)

class FrequencyBandAttention(Layer):
    """Attention over frequency bands, shared across sequence time
    steps.

    Given input X of shape (batch, timesteps, bands), the layer learns
    a
    sample-specific softmax weight vector over bands and reweights

```

X.

```
"""

def __init__(self, num_bands: int, **kwargs):
    super().__init__(**kwargs)
    self.num_bands = num_bands
    self.proj = Dense(num_bands, activation="tanh")

def call(self, inputs: tf.Tensor) -> Tuple[tf.Tensor, tf.Tensor]:
    # Summarize sequence and produce band-attention logits.
    pooled = tf.reduce_mean(inputs, axis=1)
    logits = self.proj(pooled)
    weights = tf.nn.softmax(logits, axis=-1)

    # Reweight all time steps by the same learned band profile for
    each sample.
    weighted = inputs * tf.expand_dims(weights, axis=1)
    return weighted, weights

def build_frequency_attention_lstm(seq_len: int, n_bands: int,
n_classes: int) -> Model:
    """Build Frequency-Band Attention LSTM classifier."""
    inputs = Input(shape=(seq_len, n_bands),
name="band_sequence")
    attended, band_weights = FrequencyBandAttention(n_bands,
name="freq_attention")(inputs)

    x = LSTM(64, return_sequences=True)(attended)
    x = Dropout(0.3)(x)
    x = LSTM(32)(x)
    x = Dense(32, activation="relu")(x)
    x = Dropout(0.3)(x)
    outputs = Dense(n_classes, activation="softmax",
```

```

name="class_output")(x)

model = Model(inputs=inputs, outputs=outputs,
name="FrequencyBandAttentionLSTM")
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

# Separate model to extract interpretable attention weights.
attention_model = Model(inputs=inputs, outputs=band_weights,
name="AttentionExtractor")
return model, attention_model

def build_baseline_lstm(seq_len: int, n_bands: int, n_classes: int) ->
Model:
    """Build a plain LSTM baseline without any attention
    mechanism."""
    inputs = Input(shape=(seq_len, n_bands),
name="band_sequence")
    x = LSTM(64, return_sequences=True)(inputs)
    x = Dropout(0.3)(x)
    x = LSTM(32)(x)
    x = Dense(32, activation="relu")(x)
    x = Dropout(0.3)(x)
    outputs = Dense(n_classes, activation="softmax",
name="class_output")(x)

    model = Model(inputs=inputs, outputs=outputs,
name="BaselineLSTM")
    model.compile(
        optimizer="adam",

```

```

        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model

```

```

def build_darnn(seq_len: int, n_bands: int, n_classes: int) -> Model:
    """Build a DA-RNN style classifier with feature and temporal
    attention.

```

This is adapted for classification by applying:

- input/feature attention over bands at each time step
- temporal attention over encoded sequence states

"""

```

    inputs = Input(shape=(seq_len, n_bands),
name="band_sequence")

```

Stage 1: Input attention over frequency-band features at each time step.

```

    feature_logits = Dense(n_bands,
name="darnn_feature_logits")(inputs)
    feature_weights = Softmax(axis=-1,
name="darnn_feature_attention")(feature_logits)
    attended_inputs =
Multiply(name="darnn_feature_weighted")([inputs,
feature_weights])

```

```

    encoder_seq = LSTM(64, return_sequences=True,
name="darnn_encoder")(attended_inputs)
    encoder_seq = Dropout(0.3,
name="darnn_encoder_dropout")(encoder_seq)

```

Stage 2: Temporal attention over encoder hidden states.

```

    temporal_logits = Dense(1,

```



```

name="darnn_temporal_logits")(encoder_seq)
    temporal_weights = Softmax(axis=1,
name="darnn_temporal_attention")(temporal_logits)
    context = Lambda(
        lambda tensors: tf.reduce_sum(tensors[0] * tensors[1], axis=1),
        name="darnn_context",
    )([encoder_seq, temporal_weights])

    x = Dense(32, activation="relu", name="darnn_dense")(context)
    x = Dropout(0.3, name="darnn_dense_dropout")(x)
    outputs = Dense(n_classes, activation="softmax",
name="class_output")(x)

    model = Model(inputs=inputs, outputs=outputs,
name="DARNNClassifier")
    model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model

def build_xlstm(seq_len: int, n_bands: int, n_classes: int) -> Model:
    """Build an xLSTM-inspired recurrent classifier.

    Full xLSTM from the paper is extensive; this implementation
    keeps the key
    idea of stronger gated recurrent blocks with normalization and
    residual-style
    refinement for practical EEG experiments.
    """
    inputs = Input(shape=(seq_len, n_bands),
name="band_sequence")

```

```

x = LayerNormalization(name="xlstm_input_ln")(inputs)

block1 = LSTM(64, return_sequences=True,
name="xlstm_block1_lstm")(x)
gate1 = Dense(64, activation="sigmoid",
name="xlstm_block1_gate")(block1)
block1 = Multiply(name="xlstm_block1_gated")([block1, gate1])
block1 = LayerNormalization(name="xlstm_block1_ln")(block1)
block1 = Dropout(0.3, name="xlstm_block1_dropout")(block1)

block2 = LSTM(32, return_sequences=False,
name="xlstm_block2_lstm")(block1)
gate2 = Dense(32, activation="sigmoid",
name="xlstm_block2_gate")(block2)
block2 = Multiply(name="xlstm_block2_gated")([block2, gate2])
block2 = LayerNormalization(name="xlstm_block2_ln")(block2)

x = Dense(32, activation="relu", name="xlstm_dense")(block2)
x = Dropout(0.3, name="xlstm_dense_dropout")(x)
outputs = Dense(n_classes, activation="softmax",
name="class_output")(x)

model = Model(inputs=inputs, outputs=outputs,
name="xLSTMInspiredClassifier")
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
return model

def build_model_by_name(model_name: str, seq_len: int, n_bands:

```

```

int, n_classes: int) -> Tuple[Model, Model | None]:
    """Factory to build models and optional attention-extractor
    companion models."""
    if model_name == "Baseline_LSTM":
        return build_baseline_lstm(seq_len=seq_len,
n_bands=n_bands, n_classes=n_classes), None
    if model_name == "FreqAtt_LSTM":
        return build_frequency_attention_lstm(seq_len=seq_len,
n_bands=n_bands, n_classes=n_classes)
    if model_name == "DA_RNN":
        return build_darnn(seq_len=seq_len, n_bands=n_bands,
n_classes=n_classes), None
    if model_name == "xLSTM":
        return build_xlstm(seq_len=seq_len, n_bands=n_bands,
n_classes=n_classes), None

    raise ValueError(f"Unsupported model name: {model_name}")

def build_dataset(
    gameemo_root: Path,
    fs: int,
    window_seconds: float,
    window_overlap: float,
    seq_len: int,
    seq_step: int,
    max_subjects: int | None,
    max_sequences_per_recording: int | None,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Load GAMEEMO recordings and build sequence dataset for
    classification."""
    X_parts: List[np.ndarray] = []
    y_parts: List[np.ndarray] = []
    subject_parts: List[np.ndarray] = []

```

```

        subjects = get_subject_dirs(gameemo_root,
max_subjects=max_subjects)
        if not subjects:
            raise RuntimeError(f'No subject folders found under:
{gameemo_root}')

        for subject_dir in subjects:
            subject_id = parse_subject_id(subject_dir.name)
            files = discover_preprocessed_csv_files(subject_dir)
            if not files:
                continue

            for csv_path, game_id in files:
                class_name = GAME_TO_CLASS[game_id]
                class_id = CLASS_TO_ID[class_name]

                eeg = load_eeg_csv(csv_path)
                eeg = bandpass_filter_eeg(eeg, fs=fs, low_hz=4.0,
high_hz=45.0)

                band_ts = extract_bandpower_time_series(
                    eeg=eeg,
                    fs=fs,
                    window_seconds=window_seconds,
                    window_overlap=window_overlap,
                )

                seqs = build_sequences(
                    band_ts=band_ts,
                    seq_len=seq_len,
                    seq_step=seq_step,
                    max_sequences=max_sequences_per_recording,
                )

```

```

        if seqs.size == 0:
            continue

        labels = np.full((seqs.shape[0],), class_id, dtype=np.int32)
        subjects_vec = np.full((seqs.shape[0],), subject_id,
dtype=np.int32)
        X_parts.append(seqs)
        y_parts.append(labels)
        subject_parts.append(subjects_vec)

    if not X_parts:
        raise RuntimeError("No valid EEG sequences were generated
from GAMEEMO.")

    X = np.concatenate(X_parts, axis=0)
    y = np.concatenate(y_parts, axis=0)
    subject_ids = np.concatenate(subject_parts, axis=0)
    return X, y, subject_ids

def scale_sequences(X_train: np.ndarray, X_test: np.ndarray) ->
Tuple[np.ndarray, np.ndarray]:
    """Scale band features using train-fold statistics only."""
    scaler = StandardScaler()

    X_train_2d = X_train.reshape(-1, X_train.shape[-1])
    X_test_2d = X_test.reshape(-1, X_test.shape[-1])

    X_train_scaled =
scaler.fit_transform(X_train_2d).reshape(X_train.shape)
    X_test_scaled =
scaler.transform(X_test_2d).reshape(X_test.shape)
    return
X_train_scaled.astype(np.float32),

```

```
X_test_scaled.astype(np.float32)
```

```
def get_fold_indices(
    X: np.ndarray,
    y: np.ndarray,
    subject_ids: np.ndarray,
    split_protocol: str,
    n_splits: int,
    seed: int,
    max_loso_folds: int,
) -> List[Tuple[np.ndarray, np.ndarray]]:
    """Build fold indices for either stratified CV or LOSO."""
    if split_protocol == "stratified":
        splitter = StratifiedKFold(n_splits=n_splits, shuffle=True,
random_state=seed)
        return list(splitter.split(X, y))

    if split_protocol == "loso":
        splitter = LeaveOneGroupOut()
        folds = list(splitter.split(X, y, groups=subject_ids))
        if max_loso_folds > 0:
            folds = folds[:max_loso_folds]
        return folds

    raise ValueError(f"Unsupported split protocol: {split_protocol}")

def predictive_entropy(probabilities: np.ndarray) -> np.ndarray:
    """Compute normalized predictive entropy for class
probabilities."""
    eps = 1e-12
    ent = -np.sum(probabilities * np.log(probabilities + eps), axis=1)
    return ent / np.log(probabilities.shape[1])
```

```

def compute_shap_band_importance(
    model: Model,
    X_train: np.ndarray,
    X_test: np.ndarray,
    class_names: List[str],
    band_names: List[str],
    background_size: int,
    sample_size: int,
    nsamples: int,
    random_seed: int,
) -> pd.DataFrame:
    """Compute SHAP importance aggregated at frequency-band
    level.

```

The explainer runs on flattened sequence features and the importance is

aggregated back to (time, band) then averaged over time.

"""

try:

import shap # type: ignore

except Exception as exc:

print(f'[Warn] SHAP is not available, skipping SHAP export:

{exc}')

return pd.DataFrame()

rng = np.random.default_rng(random_seed)

seq_len, n_bands = X_train.shape[1], X_train.shape[2]

bg_count = min(background_size, X_train.shape[0])

ex_count = min(sample_size, X_test.shape[0])

bg_idx = rng.choice(X_train.shape[0], size=bg_count, replace=False)

```

ex_idx      =      rng.choice(X_test.shape[0],      size=ex_count,
replace=False)

background = X_train[bg_idx].reshape(bg_count, -1)
explain = X_test[ex_idx].reshape(ex_count, -1)

def predict_fn(x_flat: np.ndarray) -> np.ndarray:
    x = x_flat.reshape((-1, seq_len, n_bands)).astype(np.float32)
    return model.predict(x, verbose=0)

explainer = shap.KernelExplainer(predict_fn, background)
shap_values      =      explainer.shap_values(explain,
nsamples=nsamples)

class_arrays: List[np.ndarray] = []
if isinstance(shap_values, list):
    class_arrays = [np.asarray(arr) for arr in shap_values]
else:
    sv = np.asarray(shap_values)
    if sv.ndim == 3 and sv.shape[2] == len(class_names):
        class_arrays = [sv[:, :, idx] for idx in
range(len(class_names))]
    elif sv.ndim == 3 and sv.shape[1] == len(class_names):
        class_arrays = [sv[:, idx, :] for idx in
range(len(class_names))]

    if len(class_arrays) != len(class_names):
        print("[Warn] Unexpected SHAP output shape, skipping SHAP
export.")
        return pd.DataFrame()

rows = []
for class_idx, class_name in enumerate(class_names):
    arr = np.asarray(class_arrays[class_idx])

```



```

        if arr.ndim != 2:
            continue

        abs_mean = np.mean(np.abs(arr), axis=0)
        abs_mean_2d = abs_mean.reshape(seq_len, n_bands)
        band_importance = abs_mean_2d.mean(axis=0)

        row = {"Class": class_name}
        for band_idx, band_name in enumerate(band_names):
            row[band_name] = float(band_importance[band_idx])
        rows.append(row)

    if not rows:
        return pd.DataFrame()

    return pd.DataFrame(rows).set_index("Class")

def save_research_figures(
    output_dir: Path,
    fold_df: pd.DataFrame,
    summary_df: pd.DataFrame,
    conf_by_model: Dict[str, pd.DataFrame],
    attention_df: pd.DataFrame,
    first_fold_histories: Dict[str, dict],
    uncertainty_summary_df: pd.DataFrame,
    shap_df: pd.DataFrame,
) -> None:
    """Save publication-ready plots for metrics and attention
    interpretation."""
    plt.style.use("seaborn-v0_8-whitegrid")

    # 1) Fold-wise comparison plot for accuracy and weighted F1.
    fig, axes = plt.subplots(1, 2, figsize=(12, 4.6), dpi=180)

```

```

for model_name in sorted(fold_df["Model"].unique()):
    subset = fold_df[fold_df["Model"] == model_name]
    axes[0].plot(subset["Fold"], subset["Accuracy"], marker="o",
linewidth=2.0, label=model_name)
    axes[1].plot(subset["Fold"], subset["F1_Weighted"],
marker="s", linewidth=2.0, label=model_name)

axes[0].set_title("Fold-wise Accuracy")
axes[0].set_xlabel("Fold")
axes[0].set_ylabel("Accuracy")
axes[0].set_ylim(0.0, 1.0)
axes[0].legend()

axes[1].set_title("Fold-wise Weighted F1")
axes[1].set_xlabel("Fold")
axes[1].set_ylabel("Weighted F1")
axes[1].set_ylim(0.0, 1.0)
axes[1].legend()

fig.tight_layout()
fig.savefig(output_dir / "figure_fold_metrics_comparison.png")
plt.close(fig)

# 2) Summary bar chart by model.
summary_pivot = summary_df.pivot(index="Model",
columns="Metric", values="Mean")

fig, ax = plt.subplots(figsize=(7.8, 4.8), dpi=180)
summary_pivot.plot(kind="bar", ax=ax)
ax.set_title("Model Comparison: Mean CV Performance")
ax.set_xlabel("Model")
ax.set_ylabel("Score")
ax.set_ylim(0.0, 1.0)
fig.tight_layout()
fig.savefig(output_dir / "figure_summary_comparison.png")

```

```

plt.close(fig)

# 3) Summed confusion matrix heatmap for each model.
for model_name, conf_df in conf_by_model.items():
    safe_name = model_name.lower().replace(" ", "_")
    fig, ax = plt.subplots(figsize=(6.2, 5.4), dpi=220)
    sns.heatmap(
        conf_df,
        annot=True,
        fmt="d",
        cmap="YlGnBu",
        cbar=True,
        square=True,
        linewidths=0.5,
        ax=ax,
    )
    ax.set_title(f'{model_name}: Summed Confusion Matrix')
    ax.set_xlabel("Predicted Label")
    ax.set_ylabel("True Label")
    fig.tight_layout()
    fig.savefig(output_dir /
f'figure_confusion_matrix_{safe_name}.png')
    if model_name == "FreqAtt_LSTM":
        fig.savefig(output_dir / "figure_confusion_matrix.png")
    plt.close(fig)

# 4) Frequency-band attention heatmap by class (proposed model
only).
if not attention_df.empty and not attention_df.isna().all().all():
    fig, ax = plt.subplots(figsize=(7.2, 4.8), dpi=220)
    sns.heatmap(
        attention_df,
        annot=True,
        fmt=".3f",

```

```

        cmap="magma",
        cbar=True,
        linewidths=0.5,
        ax=ax,
    )
    ax.set_title("Mean Frequency-Band Attention by Emotion
Class")
    ax.set_xlabel("Frequency Band")
    ax.set_ylabel("Emotion Class")
    fig.tight_layout()
    fig.savefig(output_dir / "figure_band_attention_heatmap.png")
    plt.close(fig)

```

```

fig, ax = plt.subplots(figsize=(8.2, 5.0), dpi=180)
attention_df.plot(kind="bar", ax=ax)
ax.set_title("Band Attention Profiles per Emotion Class")
ax.set_xlabel("Emotion Class")
ax.set_ylabel("Attention Weight")
ax.legend(title="Band",          ncols=len(FREQ_BANDS),
fontsize=8)
fig.tight_layout()
fig.savefig(output_dir / "figure_band_attention_bars.png")
plt.close(fig)

```

```

# 5) Uncertainty-coverage tradeoff for ensemble model.
ensemble_unc =
uncertainty_summary_df[uncertainty_summary_df["Model"] ==
"Ensemble"]
if not ensemble_unc.empty:
    fig, ax = plt.subplots(figsize=(8.2, 4.8), dpi=180)
    ax.plot(
        ensemble_unc["Threshold"],
        ensemble_unc["Coverage_Mean"],
        marker="o",

```

```

        linewidth=2.0,
        label="Coverage",
    )
    ax.plot(
        ensemble_unc["Threshold"],
        ensemble_unc["Selected_Accuracy_Mean"],
        marker="s",
        linewidth=2.0,
        label="Accuracy on Selected Samples",
    )
    ax.set_title("Ensemble Confidence Threshold vs
Coverage/Accuracy")
    ax.set_xlabel("Confidence Threshold")
    ax.set_ylabel("Score")
    ax.set_ylim(0.0, 1.0)
    ax.legend()
    fig.tight_layout()
    fig.savefig(output_dir / "figure_uncertainty_tradeoff.png")
    plt.close(fig)

```

6) SHAP band-importance heatmap (optional).

if not shap_df.empty:

```

    fig, ax = plt.subplots(figsize=(7.2, 4.8), dpi=220)
    sns.heatmap(
        shap_df,
        annot=True,
        fmt=".4f",
        cmap="viridis",
        cbar=True,
        linewidths=0.5,
        ax=ax,
    )
    ax.set_title("SHAP Band Importance by Emotion Class")
    ax.set_xlabel("Frequency Band")

```

```

ax.set_ylabel("Emotion Class")
fig.tight_layout()
fig.savefig(output_dir / "figure_shap_band_importance.png")
plt.close(fig)

# 7) Training curves for the first fold of each model.
for model_name, first_fold_history in first_fold_histories.items():
    if not first_fold_history:
        continue

    safe_name = model_name.lower().replace(" ", "_")
    fig, axes = plt.subplots(1, 2, figsize=(10.4, 4.2), dpi=180)

    axes[0].plot(first_fold_history.get("loss", []), label="Train
Loss", linewidth=2.0)
    axes[0].plot(first_fold_history.get("val_loss", []), label="Val
Loss", linewidth=2.0)
    axes[0].set_title(f"{model_name}: Fold 1 Loss")
    axes[0].set_xlabel("Epoch")
    axes[0].set_ylabel("Loss")
    axes[0].legend()

    axes[1].plot(first_fold_history.get("accuracy", []), label="Train
Accuracy", linewidth=2.0)
    axes[1].plot(first_fold_history.get("val_accuracy", []),
label="Val Accuracy", linewidth=2.0)
    axes[1].set_title(f"{model_name}: Fold 1 Accuracy")
    axes[1].set_xlabel("Epoch")
    axes[1].set_ylabel("Accuracy")
    axes[1].set_ylim(0.0, 1.0)
    axes[1].legend()

fig.tight_layout()
fig.savefig(output_dir /

```

```

f'figure_training_curves_fold1_{safe_name}.png")
    if model_name == "FreqAtt_LSTM":
        fig.savefig(output_dir / "figure_training_curves_fold1.png")
    plt.close(fig)

def evaluate_kfold(
    X: np.ndarray,
    y: np.ndarray,
    subject_ids: np.ndarray,
    split_protocol: str,
    n_splits: int,
    max_loso_folds: int,
    epochs: int,
    batch_size: int,
    seed: int,
    enable_shap: bool,
    shap_background_size: int,
    shap_sample_size: int,
    shap_nsamples: int,
    output_dir: Path,
) -> None:
    """Train/evaluate Baseline/FreqAtt models plus ensemble under
    selected protocol."""
    fold_indices = get_fold_indices(
        X=X,
        y=y,
        subject_ids=subject_ids,
        split_protocol=split_protocol,
        n_splits=n_splits,
        seed=seed,
        max_loso_folds=max_loso_folds,
    )
    n_folds = len(fold_indices)

```

```

if n_folds == 0:
    raise RuntimeError("No folds were generated for evaluation.")

model_names = [
    "Baseline_LSTM",
    "FreqAtt_LSTM",
    "DA_RNN",
    "xLSTM",
]
all_model_names = model_names + ["Ensemble"]

all_fold_rows: List[Dict[str, float | int | str]] = []
summary_rows: List[Dict[str, float | str]] = []
uncertainty_rows: List[Dict[str, float | int | str]] = []
conf_by_model_arrays: Dict[str, np.ndarray] = {
    model_name: np.zeros((len(CLASS_NAMES),
len(CLASS_NAMES))), dtype=np.int64)
    for model_name in all_model_names
}
first_fold_histories: Dict[str, dict] = {model_name: {} for
model_name in model_names}

attention_df = pd.DataFrame(index=CLASS_NAMES,
columns=list(FREQ_BANDS.keys()), dtype=float)
per_class_attention: Dict[str, List[np.ndarray]] = {name: [] for
name in CLASS_NAMES}
shap_df = pd.DataFrame()

for fold_idx, (train_idx, test_idx) in enumerate(fold_indices,
start=1):
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    X_train, X_test = scale_sequences(X_train, X_test)

```



```

model_probabilities: Dict[str, np.ndarray] = {}

for model_name in model_names:
    model, att_model = build_model_by_name(
        model_name=model_name,
        seq_len=X.shape[1],
        n_bands=X.shape[2],
        n_classes=len(CLASS_NAMES),
    )

    callbacks = [
        EarlyStopping(
            monitor="val_loss",
            patience=4,
            restore_best_weights=True,
        )
    ]

    history = model.fit(
        X_train,
        y_train,
        validation_split=0.2,
        epochs=epochs,
        batch_size=batch_size,
        verbose=0,
        callbacks=callbacks,
    )

    if not first_fold_histories[model_name]:
        first_fold_histories[model_name] = dict(history.history)

    probs = model.predict(X_test, verbose=0)
    preds = np.argmax(probs, axis=1)
    model_probabilities[model_name] = probs

```

```

        acc = accuracy_score(y_test, preds)
        f1_weighted = f1_score(y_test, preds, average="weighted",
                                zero_division=0)
        conf = confusion_matrix(y_test, preds,
                                labels=np.arange(len(CLASS_NAMES)))
        mean_entropy = float(np.mean(predictive_entropy(probs)))

        conf_by_model_arrays[model_name] += conf
        all_fold_rows.append(
            {
                "Model": model_name,
                "Fold": fold_idx,
                "Accuracy": float(acc),
                "F1_Weighted": float(f1_weighted),
                "MeanEntropy": mean_entropy,
            }
        )

        if model_name == "FreqAtt_LSTM" and att_model is not
        None:
            att_weights = att_model.predict(X_test, verbose=0)
            for class_id, class_name in enumerate(CLASS_NAMES):
                class_mask = y_test == class_id
                if np.any(class_mask):
                    per_class_attention[class_name].append(att_weights[class_mask].m
                    ean(axis=0))

            if enable_shap and shap_df.empty:
                print(" Running SHAP for FreqAtt_LSTM on first
                fold...")
                shap_df = compute_shap_band_importance(
                    model=model,

```

```

        X_train=X_train,
        X_test=X_test,
        class_names=CLASS_NAMES,
        band_names=list(FREQ_BANDS.keys()),
        background_size=shap_background_size,
        sample_size=shap_sample_size,
        nsamples=shap_nsamples,
        random_seed=seed,
    )

    print(
        f" {model_name} | fold {fold_idx}/{n_folds}: "
        f"accuracy={acc:.4f}, fl_weighted={fl_weighted:.4f}"
    )

    tf.keras.backend.clear_session()

    # Ensemble from probability averaging.
    ensemble_probs =
    np.mean(np.stack(list(model_probabilities.values()), axis=0),
    axis=0)
    ensemble_preds = np.argmax(ensemble_probs, axis=1)
    ensemble_acc = accuracy_score(y_test, ensemble_preds)
    ensemble_fl = fl_score(y_test, ensemble_preds,
    average="weighted", zero_division=0)
    ensemble_conf = confusion_matrix(y_test, ensemble_preds,
    labels=np.arange(len(CLASS_NAMES)))
    ensemble_entropy = predictive_entropy(ensemble_probs)

    conf_by_model_arrays["Ensemble"] += ensemble_conf
    all_fold_rows.append(
        {
            "Model": "Ensemble",
            "Fold": fold_idx,

```

```

        "Accuracy": float(ensemble_acc),
        "F1_Weighted": float(ensemble_f1),
        "MeanEntropy": float(np.mean(ensemble_entropy)),
    }
)

confidence = np.max(ensemble_probs, axis=1)
for threshold in np.arange(0.25, 0.96, 0.05):
    mask = confidence >= threshold
    coverage = float(np.mean(mask))
    if np.any(mask):
        selected_accuracy = float(accuracy_score(y_test[mask],
ensemble_preds[mask]))
    else:
        selected_accuracy = float("nan")

    uncertainty_rows.append(
        {
            "Model": "Ensemble",
            "Fold": fold_idx,
            "Threshold": float(np.round(threshold, 2)),
            "Coverage": coverage,
            "Selected_Accuracy": selected_accuracy,
        }
    )

fold_df = pd.DataFrame(all_fold_rows)
uncertainty_df = pd.DataFrame(uncertainty_rows)
uncertainty_summary_df = (
    uncertainty_df.groupby(["Model", "Threshold"],
as_index=False)
    .agg(
        Coverage_Mean=("Coverage", "mean"),
        Selected_Accuracy_Mean=("Selected_Accuracy", "mean"),
    )
)

```

```

        .sort_values(["Model", "Threshold"])
    )

    for model_name in all_model_names:
        model_fold_df = fold_df[fold_df["Model"] == model_name]
        for metric in ["Accuracy", "F1_Weighted", "MeanEntropy"]:
            summary_rows.append(
                {
                    "Model": model_name,
                    "Metric": metric,
                    "Mean": float(model_fold_df[metric].mean()),
                    "Std": float(model_fold_df[metric].std(ddof=0)),
                }
            )
        summary_df = pd.DataFrame(summary_rows)

    conf_by_model = {
        model_name: pd.DataFrame(conf, index=CLASS_NAMES,
columns=CLASS_NAMES)
        for model_name, conf in conf_by_model_arrays.items()
    }

    for class_name, vectors in per_class_attention.items():
        if vectors:
            mean_vec = np.mean(np.stack(vectors), axis=0)
            for band_idx, band_name in
enumerate(FREQ_BANDS.keys()):
                attention_df.loc[class_name, band_name] =
float(mean_vec[band_idx])

    output_dir.mkdir(parents=True, exist_ok=True)
    fold_df.to_csv(output_dir / "fold_metrics.csv", index=False)
    summary_df.to_csv(output_dir / "summary_metrics.csv",
index=False)

```

```

        uncertainty_df.to_csv(output_dir / "uncertainty_by_fold.csv",
index=False)
        uncertainty_summary_df.to_csv(output_dir /
"uncertainty_summary.csv", index=False)
        for model_name, conf_df in conf_by_model.items():
            safe_name = model_name.lower().replace(" ", "_")
            conf_df.to_csv(output_dir /
f"confusion_matrix_{safe_name}.csv")
            if model_name == "FreqAtt_LSTM":
                conf_df.to_csv(output_dir / "confusion_matrix.csv")

        if not attention_df.isna().all().all():
            attention_df.to_csv(output_dir /
"band_attention_by_class.csv")

        if not shap_df.empty:
            shap_df.to_csv(output_dir / "shap_band_importance.csv")

    save_research_figures(
        output_dir=output_dir,
        fold_df=fold_df,
        summary_df=summary_df,
        conf_by_model=conf_by_model,
        attention_df=attention_df,
        first_fold_histories=first_fold_histories,
        uncertainty_summary_df=uncertainty_summary_df,
        shap_df=shap_df,
    )
def build_arg_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="GAMEEMO comparison: Baseline LSTM,
FreqAtt LSTM, DA-RNN, xLSTM-inspired"
    )
    parser.add_argument(

```

```

        "--download",
        action="store_true",
        help="Download GAMEEMO using kagglehub before
training",
    )
    parser.add_argument(
        "--data-root",
        type=str,
        default="",
        help="Path to GAMEEMO root folder or its parent directory",
    )
    parser.add_argument("--fs", type=int, default=128,
help="Sampling rate (Hz)")
    parser.add_argument("--window-seconds", type=float,
default=2.0, help="Band-power window length (seconds)")
    parser.add_argument("--window-overlap", type=float,
default=0.5, help="Window overlap ratio in [0, 1)")
    parser.add_argument("--sequence-length", type=int, default=20,
help="Number of windows per sequence")
    parser.add_argument("--sequence-step", type=int, default=10,
help="Stride between sequence starts")
    parser.add_argument(
        "--split-protocol",
        type=str,
        choices=["stratified", "loso"],
        default="stratified",
        help="Evaluation split strategy: stratified or leave-one-subject-
out",
    )
    parser.add_argument("--n-splits", type=int, default=5,
help="Stratified k-fold splits")
    parser.add_argument(
        "--max-loso-folds",
        type=int,

```

```

        default=0,
        help="Cap number of LOSO folds for faster experimentation
(0=all subjects)",
    )
    parser.add_argument("--epochs", type=int, default=20,
help="Training epochs")
    parser.add_argument("--batch-size", type=int, default=64,
help="Batch size")
    parser.add_argument("--seed", type=int, default=42,
help="Random seed")
    parser.add_argument(
        "--require-gpu",
        action="store_true",
        help="Fail immediately if TensorFlow does not detect a GPU",
    )
    parser.add_argument("--max-subjects", type=int, default=0,
help="Use first N subjects for quick experiments (0=all)")
    parser.add_argument(
        "--max-sequences-per-recording",
        type=int,
        default=0,
        help="Cap sequences per recording for faster debugging (0=no
cap)",
    )
    parser.add_argument(
        "--enable-shap",
        action="store_true",
        help="Enable SHAP explainability export on first fold for
FreqAtt_LSTM",
    )
    parser.add_argument("--shap-background-size", type=int,
default=64, help="SHAP background sample count")
    parser.add_argument("--shap-sample-size", type=int, default=64,
help="SHAP explained sample count")

```



```

    parser.add_argument("--shap-nsamples", type=int, default=100,
help="SHAP KernelExplainer nsamples")
    parser.add_argument("--output-dir", type=str,
default="gameemo_results", help="Directory to save result CSV
files")
    return parser

def main() -> None:
    """Main entrypoint (Colab-safe)."""
    parser = build_arg_parser()
    args, unknown = parser.parse_known_args()

    # You can MODIFY these as needed
    if not args.data_root and not args.download:
        args.download = True # auto-download dataset

    args.epochs = args.epochs or 10
    args.batch_size = args.batch_size or 32
    args.n_splits = args.n_splits or 3 # faster for Colab

    set_reproducibility(args.seed)
    device_text =
report_and_configure_device(require_gpu=args.require_gpu)

    max_subjects = None if args.max_subjects <= 0 else
args.max_subjects
    max_seqs = None if args.max_sequences_per_recording <= 0 else
args.max_sequences_per_recording

    gameemo_root = resolve_dataset_root(args)
    print(f"Using GAMEEMO root: {gameemo_root}")

    X, y, subject_ids = build_dataset(
        gameemo_root=gameemo_root,

```

```

    fs=args.fs,
    window_seconds=args.window_seconds,
    window_overlap=args.window_overlap,
    seq_len=args.sequence_length,
    seq_step=args.sequence_step,
    max_subjects=max_subjects,
    max_sequences_per_recording=max_seqs,
)
for class_id, class_name in enumerate(CLASS_NAMES):
    print(f'- {class_name}: {(y == class_id).sum()}')

evaluate_kfold(
    X=X,
    y=y,
    subject_ids=subject_ids,
    split_protocol=args.split_protocol,
    n_splits=args.n_splits,
    max_loso_folds=args.max_loso_folds,
    epochs=args.epochs,
    batch_size=args.batch_size,
    seed=args.seed,
    enable_shap=args.enable_shap,
    shap_background_size=args.shap_background_size,
    shap_sample_size=args.shap_sample_size,
    shap_nsamples=args.shap_nsamples,
    output_dir=Path(args.output_dir),
)

```

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **137 Not Cited or Quoted 13%**
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations 0%**
Matches that are still very similar to source material
-  **2 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5%  Internet sources
- 9%  Publications
- 10%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.